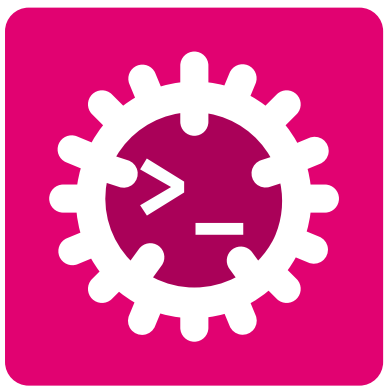


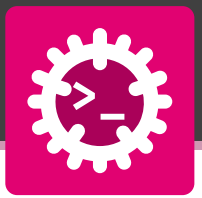
Advanced Programming

Imperative Programming

Silvan Zahno

University of Applied Sciences Western Switzerland - HES-SO Valais Wallis
Systems Engineering
Infotronics

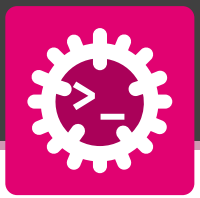
2026-08-05 - v0.1.0



Contents

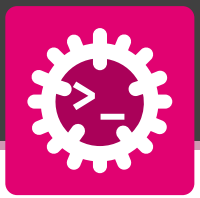
Imperative Programming 8
- Rust Playground 9
- Image explanations 10
- Naming Conventions 11
- Constants 12
- What is a Constant? 13
- Integer Boundary Constants 14
- Floating-Point Special Values (IEEE 754) 15
- Exercise - Constants 17
- Notations (Literals) 18
- Integer Literals 19
- Variables 20
- Immutable Binding - `let` 21
- Mutable Binding - `let mut` 22
- Type Inference 23

Contents



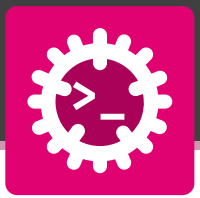
Shadowing 24
Ignoring Values: _ and _name 25
Sneak Peak: Lifetimes 26
Exercise - Variables 27
Primitive Types 28
Integer Types 29
Floating-Point Types (IEEE 754) 30
Boolean and Unit Types 31
Tuples 32
Integer Overflow 33
Exercise - Primitive Types 34
Operators 35
Arithmetic & Comparison Operators 36
Logical Operators 37
Bitwise Operators 38
Exercise - Operators 39

Contents



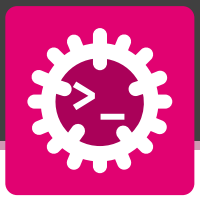
Type Conversion 40
The as Cast 41
Safe Conversion - `TryFrom` / `TryInto` and `From` / `Into` 43
Exercise - Type Conversion 44
Text Types 45
Characters 46
String Slices - `&str` 47
Owned Strings - `String` 48
`String` vs `&str` 49
String Operations 50
String Conversion 51
String Formatting `println!()`, `print!()`, `format!()` and `write!()` 52
Format Syntax Mini-Guide 55
Advanced Formatting 56
Exercise - Text Types 57
Control Flow 58

Contents



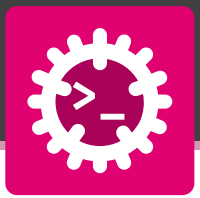
`if` 59
`if` / `else` 60
`else if` - chained conditions 61
`match` - pattern matching 62
`if let` - conditional destructuring 63
`let else` - bind or diverge 64
`loop` - infinite loop 65
`while` - condition-checked loop 66
`for` - iterator loop 67
`for` - handy iterator adapters 68
`break` / `continue` - loop control 69
Exercise - Control Flow 70
Functions 71
Function Anatomy 72
Return Values 73
Ownership & Borrowing 74

Contents



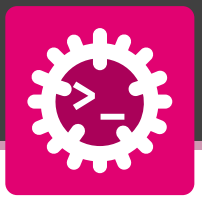
Move vs Copy 75
Passing by Value - Move & Copy 76
References - `&T` and `&mut T` 77
Closure `| |` - anonymous function 78
Exercise - Functions 79
User-Defined Types 80
Enum 81
Enum's with Data 82
Option Enum 83
Result Enum 84
Extracting Values from Option and Result with `unwrap()` 85
Extracting Values from Option and Result with `expect()` 86
Be Careful with `unwrap()` and `expect()` 87
Struct 90
Exercise - Option and Result 91
Ownership and Borrowing 92

Contents



Ownership 93
Ownership - Move 94
Ownership - Copy 95
Ownership - Passing Values to Functions 96
Ownership - Borrowing 97
Ownership - Mutable Borrowing 98
Ownership - Borrowing Rules 99
Lifetimes 100
Lifetime Annotations `'a` 102
Exercise - Ownership and Borrowing 103
Composite Types 104
Composite Types 105
Arrays `[T; N]` 106
Multidimensional Arrays `[[T; N]; M]` 107
Vector `Vec<T>` 108
`HashMap<K, V>` 110

Contents

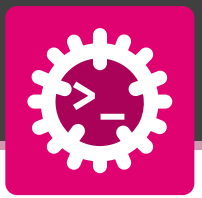


BTreeMap<K, V> 112
HashSet<T> 113
BTreeSet<T> 114
VecDeque 115
Which Collection Should I Use? 116
Collection Overview 117
Container Overview 119
Exercise - Composite Types 120
Glossary & Bibliography 121

Imperative Programming

A hands-on, step-by-step approach to coding

Rust Playground



During lectures or for small experiments use the Rust Playground - an online editor with instant compilation and sharing.

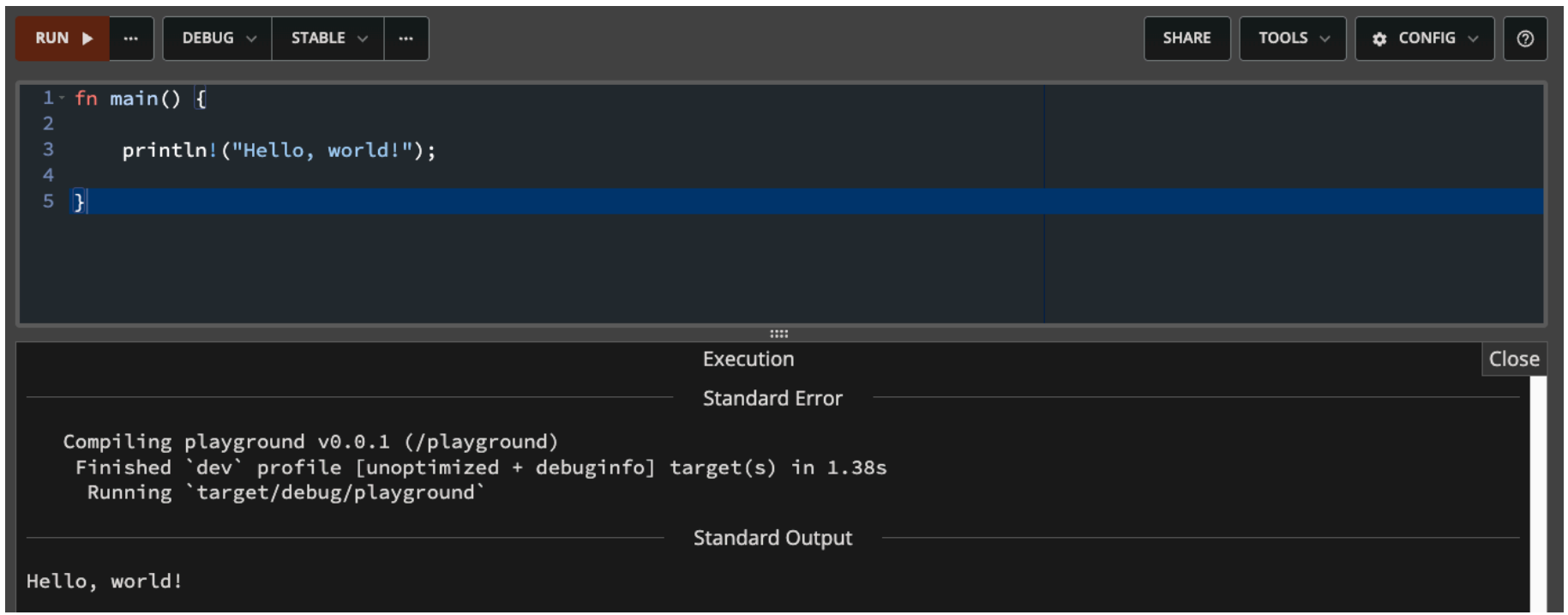
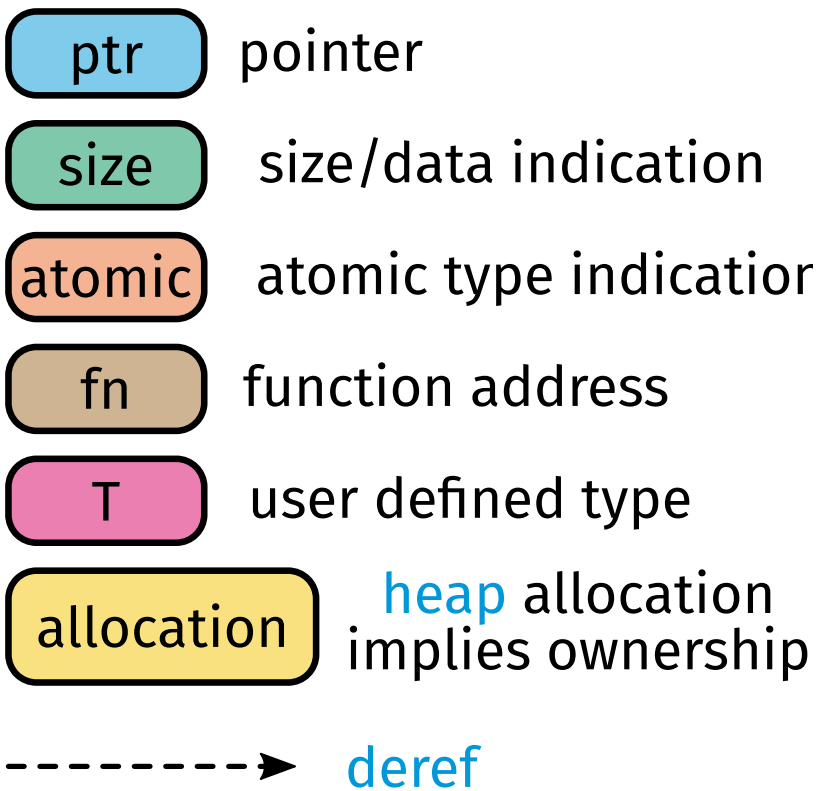


Image explanations



Naming Conventions

Rust uses different naming styles for different language elements. The compiler accepts many names, but `rustfmt` and `clippy` expect the common Rust style.

Element	Convention	Example
Variable	`snake_case`	`sensor_value`
Function	`snake_case`	`read_sensor()`
Module	`snake_case`	`sensor_bus`
Constant	`SCREAMING_SNAKE_CASE`	`MAX_RETRIES`
Static	`SCREAMING_SNAKE_CASE`	`APP_NAME`
Type	`UpperCamelCase`	`SensorValue`
Struct	`UpperCamelCase`	`TemperatureSensor`
Enum	`UpperCamelCase`	`SensorState`
Enum variant	`UpperCamelCase`	`OutOfRange`
Trait	`UpperCamelCase`	`Readable`
Generic type	`UpperCamelCase`	`T`, `Value`

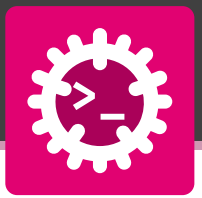
Names are part of the program design.

- Good names reduce comments.
- Names should communicate intent.

Constants

Values known at compile time

What is a Constant?



Definition

A **constant** is a name permanently bound to a **compile-time value**.

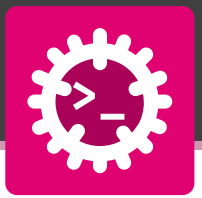
- Declared with `const` keyword **anywhere**
- Type annotation is **mandatory**
- Name uses `SCREAMING_SNAKE_CASE`
- Valid for the **entire** program lifetime
- **Inlined** by the compiler - zero runtime cost

`const` **vs** `let`

`const`	`let`
Compile-time value	Runtime binding
Always immutable	Immutable by default
No memory address	Lives on the stack
Any scope	Block scope only

```rust
// Type annotation is required
const MAX_SCORE:  u32  = 100_000;
const GRAVITY:    f64  = 9.81;
const APP_NAME:   &str = "RustApp";
const MAX_RETRIES: u8  = 3;

// Computed constant expression
const SECS_PER_DAY: u64 = 60 * 60 * 24;

fn main() {
    println!("{APP_NAME}");
    println!("max score: {MAX_SCORE}");
    println!("seconds/day: {SECS_PER_DAY}");

    // Constants can be used in other constants
    const DAYS_PER_YEAR: u64 = 365;
    const SECS_PER_YEAR: u64 = SECS_PER_DAY * DAYS_PER_YEAR

    println!("seconds/year: {SECS_PER_YEAR}");
}
```

Integer Boundary Constants



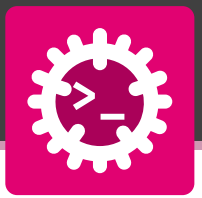
Min and Max values

Every integer type exposes `MIN`, `MAX` and `BITS` as associated constants:

```rust
fn main() {
  println!("u{}   : {}..={}", u8::BITS, u8::MIN, u8::MAX);
  println!("u{}   : {}..={}", u16::BITS, u16::MIN, u16::MAX);
  println!("u{}   : {}..={}", u32::BITS, u32::MIN, u32::MAX);
  println!("u{}   : {}..={}", u64::BITS, u64::MIN, u64::MAX);
  println!("usize : {}..={}", usize::MIN, usize::MAX);
  println!("i{}   : {}..={}", i8::BITS, i8::MIN, i8::MAX);
  println!("i{}   : {}..={}", i16::BITS, i16::MIN, i16::MAX);
  println!("i{}   : {}..={}", i32::BITS, i32::MIN, i32::MAX);
  println!("i{}   : {}..={}", i64::BITS, i64::MIN, i64::MAX);
  println!("isize : {}..={}", isize::MIN, isize::MAX);
}
```

Type	Min	Max	Bits
u8	0	255	8
u16	0	65 535	16
u32	0	4.3×10^9	32
u64	0	1.8×10^{19}	64
u128	0	$\approx 3.4 \times 10^{38}$	128
usize	0	platform	32/64
i8	−128	127	8
i16	−32 768	32 767	16
i32	-2.1×10^9	2.1×10^9	32
i64	-9.2×10^{18}	9.2×10^{18}	64
i128	$\approx -1.7 \times 10^{38}$	$\approx 1.7 \times 10^{38}$	128
isize	platform	platform	32/64

Floating-Point Special Values (IEEE 754)



Mathematical constants - std::f64::consts:

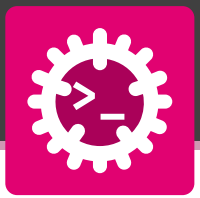
π, e, $\sqrt{2}$, $\ln(10)$, **etc.**

```rust
use std::f64::consts;

fn main() {
  println!("π         = {:.10}", consts::PI);
  println!("τ = 2π    = {:.10}", consts::TAU);
  println!("e         = {:.10}", consts::E);
  println!("√2        = {:.10}", consts::SQRT_2);
  println!("ln 2      = {:.10}", consts::LN_2);
  println!("ln 10     = {:.10}", consts::LN_10);
  println!("1/π       = {:.10}", consts::FRAC_1_PI);
  println!("1/√2      = {:.10}", consts::FRAC_1_SQRT_2);
  println!("log10(2) = {:.10}", consts::LOG10_2);
}
```

```
π         = 3.1415926536
τ = 2π    = 6.2831853072
e         = 2.7182818285
√2        = 1.4142135624
ln 2      = 0.6931471806
ln 10     = 2.3025850930
1/π       = 0.3183098862
1/√2      = 0.7071067812
log10(2) = 0.3010299957
```

```rust
use std::f64::consts;

fn main() {
  let radius = 4.234;
  let diameter = 2.0 * radius;
  let area = consts::PI * radius * radius; // πr²
  println!("r={radius} d={diameter} area={area:.3}");
}
```

Floating-Point Special Values (IEEE 754)

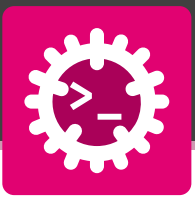


```rust
use std::f64::consts;
fn main() {
    let inf     = f64::INFINITY;
    let neg_inf = f64::NEG_INFINITY;
    let nan     = f64::NAN;

    println!(" ∞  = {inf}");                  //  inf
    println!("-∞  = {neg_inf}");              // -inf
    println!("NaN = {nan}");                  //  NaN

    // Arithmetic with special values
    println!("{}", 1.0_f64 / 0.0);            //  inf
    println!("{}", -1.0_f64 / 0.0);           // -inf
    println!("{}", 0.0_f64 / 0.0);            //  NaN

    // NaN is never equal - even to itself!
    println!("{}", f64::is_nan(f64::NAN));    // true
    println!("{}", f64::NAN.is_nan());        // true
    println!("{}", inf.is_infinite());        // true
    println!("{}", 42.0_f64.is_finite());     // true
}
```

$\infty, -\infty, \text{NaN}$

```
 ∞  = inf
-∞  = -inf
 NaN = NaN
inf
-inf
NaN
true
true
true
true
```

Exercise - Constants



Using **only** `const` declarations (no `let` bindings), build a tiny time budget:

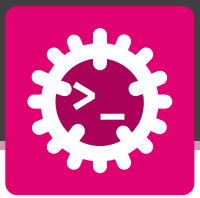
1. Declare the base constants `SECS_PER_MINUTE`, `MINS_PER_HOUR`, `HOURS_PER_DAY` and `DAYS_PER_YEAR`.
2. Derive `SECS_PER_YEAR` as a **computed constant expression** from the constants above.
3. In `main`, print `SECS_PER_YEAR` and report whether it fits in a `u16` and in a `u32`, using the associated boundary constants `u16::MAX` and `u32::MAX`.

Use `SCREAMING_SNAKE_CASE` names and pick an integer type wide enough to hold the result.

Notations (Literals)

Expressing values directly in source code

Integer Literals



Decimal, Hex, Octal, Binary, Byte

Supports five **numeric bases** for integer literals

```
fn main() {
  let decimal  = 98_222;      // base 10
  let hex      = 0xFF;        // base 16, prefix 0x
  let octal    = 0o77;        // base 8,  prefix 0o
  let binary   = 0b1111_0000; // base 2,  prefix 0b
  let byte: u8 = b'A';        // ASCII byte literal

  println!("decimal : {decimal}"); // 98222
  println!("hex     : {hex}");     // 255
  println!("octal   : {octal}");   // 63
  println!("binary  : {binary}");  // 240
  println!("byte    : {byte}");    // 65

  // All represent the same value
  assert_eq!(0xFF_u8, 0b1111_1111_u8);
  assert_eq!(0o377_u16, 255_u16);
}
```

Number separator _ improves readability:

```
let million: u64    = 1_000_000;
let pi_approx: f64  = 3.141_592_653;
let flags: u8       = 0b0000_1111;
let ip_mask:u32     = 0xFF_FF_00_00;
```

Type suffixes make the type explicit:

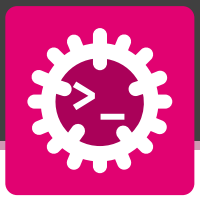
```
let a: u8   = 57;      // unsigned 8-bit
let b: i32  = -100;    // signed 32-bit
let c: f32  = 1.5;     // 32-bit float
let d: i64  = 1_000;   // large signed integer
let e       = 57_i16;  // explicit i16 suffix
```

Default types when no suffix is given: integer `i32`, float `f64`, byte `u8`

Variables

Bindings: immutable by default

Immutable Binding - `let`



In Rust, bindings are **immutable by default**. The compiler **enforces** this at compile time:

```rust
fn main() {
  let x = 5;
  println!("x = {x}");    // x = 5
  // This does NOT compile:
  x = 6;
}
```

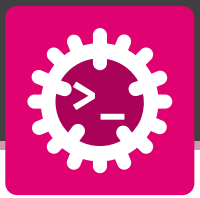
```
error[E0384]: cannot assign twice to
              immutable variable `x`
 --> src/main.rs:4:5
2 |     let x = 5;
  |         -
  |         |
  |         first assignment to `x`
  |         help: consider making this binding mutable: `mut x`
4 |     x = 6;
  |     ^^^^^ cannot assign twice to immutable variable
```

Why immutable by default?

- Prevents accidental mutation bugs
- Enables safe concurrent access
- Communicates intent explicitly
- Allows aggressive compiler optimizations
- Forces deliberate design choices

The compiler is your **friend** - it gives clear error messages and suggestions to fix issues.

Mutable Binding - `let mut`



Add `mut` to allow reassignment. But **start immutable, add `mut` only when needed.**

```
struct Sensor { value: f64, unit: &'static str }
fn main() {
  // Mutable variable - can be reassigned
  let mut count = 0_u32;
  count += 1;
  println!("count = {count}");  // count = 1

  // Mutable bindings can also hold complex types
  let mut name = String::from("Alice");
  name.push_str(" Smith");
  println!("{name}");           // Alice Smith

  // Mutable structs: `mut` applies to all fields
  let mut s = Sensor { value: 20.0, unit: "°C" };
  // Simulate readings
  s.value += 3.5;
  s.value -= 1.2;
  println!("{:.1} {}", s.value, s.unit); // 22.3 °C
}
```

Real-world example - accumulator loop:

```
fn main() {
  let readings = [17.3, 19.1];
  let mut sum = 0.0_f64;

  for &r in &readings {
    sum += r;
    println!("reading: {r:.1}, sum: {sum:.1}");
  }

  let avg = sum / readings.len() as f64;
  println!("average: {avg:.1}");
}
```

```
reading: 17.3, sum: 17.3
reading: 19.1, sum: 36.4
average: 18.2
```

Type Inference

Rust infers types from **context** - no annotation needed when unambiguous:

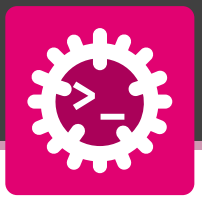
Basic inference from value:

```rust
fn main() {
    let a = 42;          // i32   (integer default)
    let b = 3.14;        // f64   (float default)
    let c = true;        // bool
    let d = 'z';         // char
    let e = "hello";     // &str
    let f = 42_u8;       // u8
    let g = 42_i128;     // i128
    // Inferred from another binding
    let x = 100;
    let y = x;           // i32 - same as x
    // Inferred from arithmetic context
    let sum = a + y;     // i32

    println!("{a} {b} {c} {d} {e} {f} {g} {sum}");
}
```

Same code with explicit type annotations:

```rust
fn main() {
    let a: i32  = 42;
    let b: f64  = 3.14;
    let c: bool = true;
    let d: char = 'z';
    let e: &str = "hello";
    let f: u8   = 42;
    let g: i128 = 42;
    let x: i32 = 100;
    let y: i32 = x;
    let sum: i32 = a + y;

    println!("{a} {b} {c} {d} {e} {f} {g} {sum}");
}
```

Shadowing



Shadowing lets you reuse a name with a **new binding** - even with a different type:

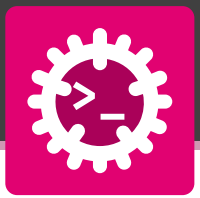
```
fn main() {
  let x = 5;          // x: i32 = 5
  let x = x + 1;      // x: i32 = 6  (new binding)
  {
    let x = x * 2;    // x: i32 = 12 (inner scope)
    println!("inner x = {x}"); // 12
  }
  println!("outer x = {x}");   // 6 (outer still 6)

  // Shadow with a completely different TYPE
  let spaces = "   ";           // &str
  let spaces = spaces.len();    // usize = 3
  println!("spaces = {spaces}");
}
```

Shadowing vs `mut`:

```
// With mut: same type, reassignable
let mut y = 5;
y = y + 1;     // ok - same type i32
// y = "six"; // ERROR: type mismatch

// With shadowing: can change type!
let z = 5;
let z = z + 1;   // new binding, still i32
let z = "six";   // new binding, now &str
println!("{z}");
```

What prints the program out?

Ignoring Values: _ and _name



Two ways to say “I don’t need this” - keep the value but silence the warning, or throw it away:

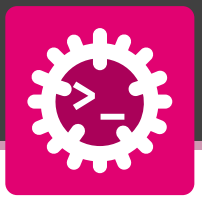
```
fn main() {
  // _name: BOUND but unused → no warning
  let _guard = setup(); // kept alive, name silenced
  let count = 5;        // ⚠ warning: unused `count`
  let _count = 5;       // no warning

  // bare _ : NOT bound → value thrown away
  let _ = returns_result(); // ignore the Result
  let (x, _) = (1, 2);      // keep x=1, drop the 2

  for _ in 0..3 {           // loop 3×
    println!("tick");       // index not used
  }
}
```

_name **vs** _ - the key difference:

- `_name` - a **real binding**; the value **lives** to end of scope, the leading _ only mutes the “unused” warning
- `_` - **not a binding**; the value is **dropped immediately**, nothing is named
- So `let _g = guard();` **holds** the guard; `let _ = guard();` drops it **now**
- _ also works in patterns: `match`, `if let`, tuples, closures - “match anything, bind nothing”

When does the `Guard` get dropped: with `let _g =` versus `let _ =`?

Sneak Peak: Lifetimes

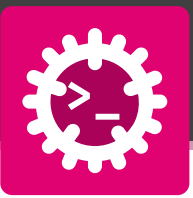


Lifetime is a compile-time concept that tracks **how long references are valid**. It ensures memory safety without a garbage collector.

In the example below, the inner x shadows the outer x, but they have different lifetimes - the inner x only exists within the inner block and is dropped when the block ends. The outer x remains unaffected and valid throughout the entire `main` function.

```
fn main() {
    let x = 5;                    // ----------------------------------+-- b'
    let x = x + 1;                //                                   |
    {                             //                                   |
      let x = x * 2;              // --+-- a'                          |
      println!("inner x = {x}"); //   |                               |
    }                             // --+-- a' ends, inner x dropped    |
    println!("outer x = {x}");    //                                   |
}                                 // ----------------------------------+-- b' ends, outer x dropped
```

Exercise - Variables



Write a small `main` that exercises bindings and shadowing:

1. Declare an immutable `let level = 1`, then explain in one comment why `level = 2;` would **not compile**
2. Print value of variable `level` with `println!("{level}")`
3. Declare a mutable `let mut score = 0`, then add `10` to it **twice**
4. Remove the `warning: variable score is assigned to, but never used`
5. Starting from the string `"  42  "`, use **shadowing** to reuse the same name: first trim it with `.trim()`, then parse it into an `i32` with `.parse().unwrap()`
6. Print the value of the `i32` variable

Lean on **type inference** and only annotate a type where the compiler needs it.

Primitive Types

Scalar and compound built-in types

Integer Types

Type	Min	Max	Typical use
u8	0	255	bytes, RGB, ASCII
u16	0	65 535	network ports
u32	0	4.3×10^9	file sizes, colours
u64	0	1.8×10^{19}	large counters, hashes
u128	0	≈ 3.4e38	UUIDs, checksums
usize	0	platform	lengths, indices
i8	−128	127	small deltas, sensor raw
i16	−32 768	32 767	audio samples
i32	-2.1×10^9	2.1×10^9	default integer
i64	-9.2×10^{18}	9.2×10^{18}	timestamps, large IDs
i128	≈ −1.7e38	≈ 1.7e38	cryptography, finance
isize	platform	platform	signed index offsets

isize / usize match the **pointer width** and are platform dependent (32 or 64 bit):

```rust
fn main() {
  let v = vec![10, 20, 30, 40];

  // len() returns usize
  let len: usize = v.len();
  println!("last: {}", v[len - 1]); // 40

  // Signed arithmetic on indices
  let offset: isize = -2;
  let i = (len as isize + offset) as usize;
  println!("v[{}] = {}", i, v[i]); // v[2] = 30

  // Platform size
  println!("usize = {} bits", usize::BITS);
  // 64 on modern machines
}
```

Floating-Point Types (IEEE 754)

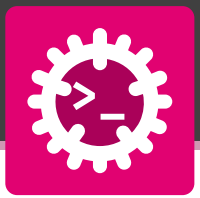
Type	Width	Precision	Range (abs)
f32	32 bit	7 decimal dig	1.2e-38 to 3.4e38
f64	64 bit	15 decimal dig	2.2e-308 to 1.8e308

```rust
fn main() {
  // f64 is the default float type
  let y: f64 = 3.14159265358979;

  let area = std::f64::consts::PI * 5.0_f64.powi(2);
  println!("area: {area:.4}");     // area: 78.5398

  // Precision difference
  let a: f32 = 0.1 + 0.2;
  let b: f64 = 0.1 + 0.2;
  println!("f32: {a:.20}"); // 0.30000001192092895508
  println!("f64: {b:.20}"); // 0.30000000000000004441
}
```

Common math methods:

```rust
let x = 2.0_f64;

println!("{}", x.sqrt());    // 1.4142135623730951
println!("{}", x.powi(10));  // 1024.0  (integer exp)
println!("{}", x.powf(0.5)); // 1.4142135623730951
println!("{}", x.ln());      // 0.6931471805599453
println!("{}", x.log2());    // 1.0
println!("{}", x.log10());   // 0.30102999566398114
println!("{}", x.sin());     // 0.9092974268256817
println!("{}", x.cos());     // -0.4161468365471424
println!("{}", x.abs());     // 2.0
println!("{}", (-3.7_f64).ceil());   // -3.0
println!("{}", (-3.7_f64).floor());  // -4.0
println!("{}", (-3.7_f64).round());  // -4.0
println!("{}", (-3.7_f64).trunc());  // -3.0
```

Boolean and Unit Types



`bool` - exactly 1 byte, values `true` or `false`:

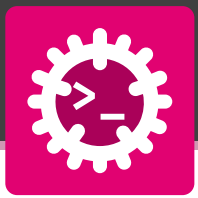
```rust
fn main() {
    let t: bool = true;
    let f       = false; // type inferred

    // Produced by comparison operators
    let is_adult = 18 ≥ 18;          // true
    let is_zero  = 42 == 0;          // false
    let in_range = 5 < 10 && 10 < 20; // true

    // Used in control flow
    if is_adult {
        println!("welcome");
    }

    // Casting to integer
    println!("{}", true  as u8);    // 1
    println!("{}", false as u8);    // 0
}
```

Unit type `()` - empty tuple, zero bytes, “no value”:

```rust
// Functions without -> return ()
fn greet(name: &str) {
    println!("Hello, {name}!");
}

fn main() {
    let result: () = greet("Alice");
    println!("{:?}", result);   // ()
}
```

:? is the debug formatter, which prints values in a human-readable form. For (), it simply prints ().

Tuples



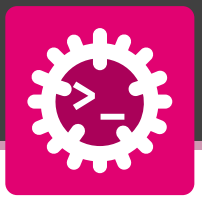
Tuple - ordered, fixed-size group of **heterogeneous** values:

```
fn main() {
  // Type annotation: (T1, T2, T3, ...)
  let point: (i32, i32)       = (10, 20);
  let color: (u8, u8, u8)     = (255, 128, 0); // RGB
  let mixed: (i32, f64, bool) = (42, 3.14, true);

  // Access by zero-based index (.0, .1, ...)
  println!("{}", point.0);    // 10
  println!("{}", color.2);    // 0
  println!("{}", mixed.1);    // 3.14

  // Tuples are Copy if all fields are Copy
  let a = point;
  let b = point;  // both valid
  println!("{} {}", a.0, b.0); // 10 10
}
```

Destructuring - unpack a tuple into named variables:

```
fn main() {
  let point = (10, 20);
  let color = (255_u8, 128_u8, 0_u8);
  let mixed = (42, 3.14, true);

  // bind all fields
  let (x, y)      = point;
  let (r, g, b)   = color;

  // _ discards a field
  let (n, _, ok)  = mixed;
  println!("{x},{y}  r={r} ok={ok} n={n}");

  // Nested tuples
  let nested = ((1, 2), (3, 4));
  println!("{}", nested.0.1);  // 2  (first tuple, second
field)
}
```

Integer Overflow



Default behaviour depends on build mode. Use explicit methods for full control:

```rust
fn main() {
  let x: u8 = 255;

  let bad = x + 1; // debug build: panics on overflow
  // release build: wraps silently (255 + 1 == 0)

  // ── Explicit overflow strategies ──
  // wrapping_*: always wraps (modular arithmetic)
  println!("{}", x.wrapping_add(1));      // 0
  println!("{}", x.wrapping_add(10));     // 9

  // saturating_*: clamps to boundary
  println!("{}", x.saturating_add(1));    // 255
  println!("{}", 0_u8.saturating_sub(5)); // 0

  // checked_*: returns Option
  println!("{:?}", x.checked_add(1));    // None
  println!("{:?}", x.checked_add(0));    // Some(255)
}
```

When does overflow matter?

- Embedded systems with small integer buffers
- Network protocol byte counters
- Cryptographic arithmetic
- Loop index arithmetic

Summary of overflow methods:

Method	Behaviour
`wrapping_add`	wrap around (modular)
`saturating_add`	clamp to `MIN`/`MAX`
`checked_add`	return `Option<T>`
`overflowing_add`	return `(T, bool)`

All four families exist: `_add`, `_sub`, `_mul`, `_div`, `_shl`, `_shr`, `_neg`.

Exercise - Primitive Types

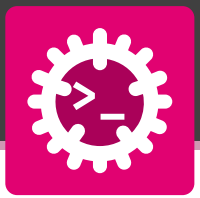
Read the code below (don't compile it yet) and spot the **type errors**. For each line, say **why** the compiler rejects it and how you would fix it.

```
fn main() {
  let a: u8 = 300;
  let b: u8 = 10;
  let sum: u16 = a + b;

  let ratio = 7 / 2.0;

  let flag: bool = 1;

  let point = (1, 2.0);
  let z: i32 = point.1;
}
```

Operators

Arithmetic, Comparison, Logical, and Bitwise

Arithmetic & Comparison Operators



Arithmetic - +, −, *, /, %:

```rust
fn main() {
  let a: i32 = 17;
  let b: i32 = 5;

  println!("{a} + {b} = {}", a + b);  // 22
  println!("{a} - {b} = {}", a - b);  // 12
  println!("{a} * {b} = {}", a * b);  // 85
  println!("{a} / {b} = {}", a / b);  // 3 ← int
  println!("{a} % {b} = {}", a % b);  // 2

  // Integer division truncates toward zero
  println!("{}", -7_i32 / 2);  // -3  (not -4!)
  println!("{}", -7_i32 % 2);  // -1

  // Compound assignment
  let mut n = 10;
  n += 5;  n -= 3;  n *= 2;  n /= 4;  n %= 4;
  println!("{n}"); // 2
}
```

Comparison - return bool:

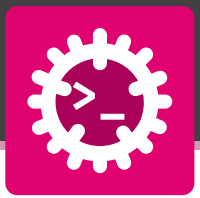
```rust
fn main() {
  let x = 42;
  println!("{}", x == 42);   // true  (equal)
  println!("{}", x ≠ 42);   // false (not equal)
  println!("{}", x >  10);   // true  (greater than)
  println!("{}", x < 100);   // true  (less than)
  println!("{}", x ≥ 42);   // true  (greater or
equal)
  println!("{}", x ≤ 42);   // true  (less or equal)
}
```

Operator precedence:

Highest ⇨ Lowest (*, /, %),

(+, −), (<<, >>), (&), (^), (|), (==, ≠, <, >, ≤, ≥), (&&), (||)

Logical Operators



Logical Operators - work on `bool` values.

&& AND, || OR, ^ (XOR) and ! NOT:

```
fn main() {
  let a: bool = true;
  let b: bool = false;

  println!("{}", a && b); // false  (AND)
  println!("{}", a || b); // true   (OR)
  println!("{}", a ^ b);  // true   (XOR)
  println!("{}", !a);     // false  (NOT)
}
```

```
  fn main() {
    // && eval from left to right
    // RHS not evaluated if result is known
    let x: i32 = 0;
    // Safe: 10/x never runs when x == 0
    if x != 0 && 10 / x > 1 {
        println!("big quotient");
    } else {
        println!("x is zero, skipped division");
    }

    // || RHS not evaluated if LHS is true
    fn expensive() -> bool { println!("called!"); true }
    if true || expensive() {
        println!("short-circuited");
    }
    // "short-circuited" - expensive() never ran
}
```

Bitwise Operators

Bitwise operators - work on integer bits:

```rust
fn main() {
  let a: u8 = 0b1100_1010; // 202
  let b: u8 = 0b1010_1100; // 172

  println!("{:08b}", a & b); // 10001000 AND
  println!("{:08b}", a | b); // 11101110 OR
  println!("{:08b}", a ^ b); // 01100110 XOR (toggle)
  println!("{:08b}", !a);    // 00110101 NOT (bitflip)

  // Shift operators
  let flags: u8 = 0b0000_0001;
  println!("{:08b}", flags << 3); // 00001000 (×8)
  println!("{:08b}", flags >> 1); // 00000000 (÷2)
}
```

Practical: hardware register flags

```rust
fn main() {
  const LED_ON:    u8 = 0b0000_0001;
  const MOTOR_ON:  u8 = 0b0000_0010;
  const ALARM_ON:  u8 = 0b0000_0100;

  let mut reg: u8 = 0b0000_0000;

  // set bits   → 0b0000_0011
  reg |= LED_ON | MOTOR_ON;

  // flip bits  → 0b0000_0111
  reg ^= ALARM_ON;

  // clear bits → 0b0000_0011
  reg &= !ALARM_ON;

  // 0b00000011 (LED & MOTOR)
  println!("0b{:08b}", reg);
}
```

Exercise - Operators

Trace the code by hand and write down the **output value** printed by each line. Mind **integer division**, **bitwise** operators and **operator precedence**.

```
fn main() {
  let a = 17;
  let b = 5;
  println!("{}", a / b);                    // 1
  println!("{}", a % b);                    // 2
  println!("{}", a / b * b);                // 3

  let x = 0b1010;                           // 10
  let y = 0b0110;                           // 6
  println!("{}", x & y);                    // 4
  println!("{}", x | y);                    // 5
  println!("{}", x ^ y);                    // 6
  println!("{}", x << 1);                   // 7

  println!("{}", true && false || !false); // 8
  println!("{}", 3 + 4 * 2 > 10);           // 9
}
```

Type Conversion

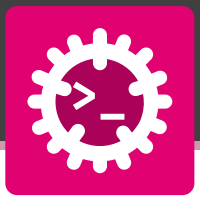
Explicit, safe, and idiomatic conversions

The as Cast

as performs **numeric casts** - fast but potentially **lossy**:

```rust
fn main() {
    // Widening
    // (always safe)
    let small: u8  = 200;
    let wide: u32  = small as u32;
    println!("{wide}");              // 200

    // Narrowing
    // (truncates bits - potential loss!)
    let big: u32  = 300;
    let tiny: u8  = big as u8;      // 300 - 256 = 44
    println!("{tiny}");              // 44

    // Float → integer
    // (truncates toward zero, no round)
    let f: f64 = 3.99;
    let i: i32 = f as i32;
    println!("{i}");                 // 3

    // Negative float → unsigned: saturates
    let neg: f64 = -1.5;
    let u: u32   = neg as u32;
    println!("{u}");                 // 0

    // char ↔ u32
    let c = 'A';
    println!("{}", c as u32);        // 65
    println!("{}", 66_u8 as char); // B
}
```

The as Cast



Why is as dangerous?

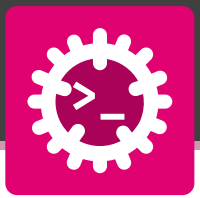
```
// Example: reading sensor data
fn read_sensor() -> u32 { 300 }  // returns 300

fn process() {
    let raw: u32 = read_sensor();

    // Silent data loss - no warning!
    let byte: u8 = raw as u8;   // 44, not 300!

    // Use in protocol field -> corrupted packet
    println!("sending byte: {byte}");
}
```

```
300 = 0b00000001_00101100
                 ^^^^^^^^
                 keep these 8 bits
0b00101100 = 44
```

Rule of thumb for as:

Situation	Recommendation
Widening (u8 → u32)	as is always safe
Float ↔ integer	as is OK (truncates)
Narrowing (u32 → u8)	prefer `try_into()`
Pointer casts	as - systems code only

Safe Conversion - TryFrom / TryInto and From / Into



Fallible - returns Result, never silently loses data:

```rust
use std::convert::TryFrom;
use std::convert::TryInto;

fn main() {
    // TryFrom: explicit, static method
    let big: i32 = 300;
    match u8::try_from(big) {
      Ok(v)  => println!("ok: {v}"),
      Err(e) => println!("error: {e}"),
    } // error: out of range integral type conversion

    // TryInto: method syntax (often preferred)
    let ok: i32 = 200;
    let result: Result<u8, _> = ok.try_into();
    println!("{:?}", result);  // Ok(200)

    let bad: i32 = -5;
    let result: Result<u8, _> = bad.try_into();
    println!("{:?}", result); // Err(TryFromIntError(()))
}
```

Infallible - From / Into (always succeeds):

```rust
// From: explicit construction
let s = String::from("hello");  // &str → String
let n = i64::from(42_i32);      // i32  → i64

// Into: derived automatically from From
let s: String = "world".into(); // &str → String
let n: i64    = 42_i32.into();  // i32  → i64

// Custom conversion
struct Celsius(f64);
struct Kelvin(f64);

impl From<Celsius> for Kelvin {
  fn from(c: Celsius) -> Self {
    Kelvin(c.0 + 273.15)
  }
}

let boiling = Kelvin::from(Celsius(100.0));
println!("{:.2} K", boiling.0);  // 373.15 K

// Into is auto-derived
let freezing: Kelvin = Celsius(0.0).into();
println!("{:.2} K", freezing.0); // 273.15 K
```

Exercise - Type Conversion

Trace each as cast and write down the **output value**. Watch for **narrowing**, **truncation toward zero** and **saturation**.

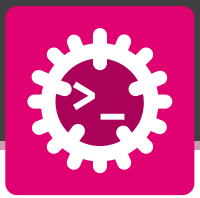
```
fn main() {
  println!("{}", 300_i32 as u8);   // 1
  println!("{}", 3.99_f64 as i32); // 2
  println!("{}", -1.5_f64 as u32); // 3
  println!("{}", 'A' as u32);      // 4
  println!("{}", 66_u8 as char);   // 5
  println!("{}", 255_u8 as i8);    // 6
  println!("{}", -1_i8 as u8);     // 7
}
```

as never fails - it silently reinterprets or clamps. Prefer `TryFrom`/`TryInto` when loss must be caught.

Text Types

char, &str, String

Characters



`char` - a single Unicode Scalar Value (USV), always 4 bytes wide:

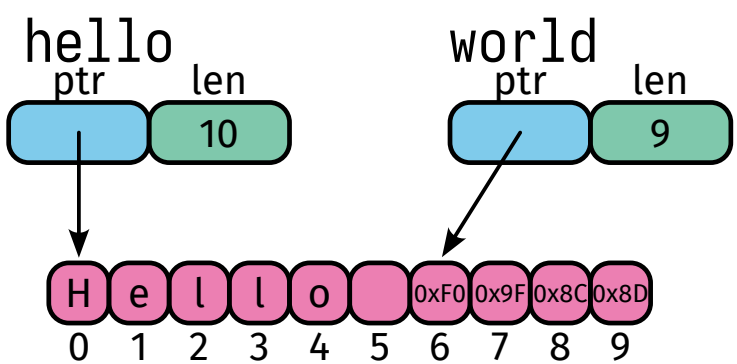
```rust
fn main() {
  let letter: char = 'A';
  let emoji:  char = '🦀';       // Ferris the crab!
  let accent: char = 'é';        // accented letter
  let japanese: char = '日';     // "day" in Japanese
  let ascii_a: i32 = 'A' as i32; // 65 - see ASCII
table

  // Escape Characters
  let newline: char         = '\n';
  let carriage_return: char = '\r';
  let tab: char             = '\t';
  let quote: char           = '\'';
  let backslash: char       = '\\';
  let 7bit-unicode: char = '\u{7F}';      // DEL
  let 8bit-unicode: char = '\u{1F600}';   // 😀
  let 24bit-unicode: char = '\u{10FFFF}';

  // Classification
  println!("{}", 'a'.is_alphabetic());   // true
  println!("{}", '5'.is_ascii_digit());  // true
  println!("{}", '5'.is_alphanumeric()); // true
  println!("{}", '²'.is_numeric());      // true
  println!("{}", ' '.is_whitespace());   // true
  println!("{}", 'A'.is_uppercase());    // true
  println!("{}", 'z'.is_lowercase());    // true
  println!("{}", 'A'.is_ascii());        // true

  // Case conversion
  println!("{}", 'a'.to_uppercase()); // A (it)
  println!("{}", 'Z'.to_lowercase()); // z (it)

  // Digit conversion
  println!("{:?}", 'F'.to_digit(16)); // Some(15)
  println!("{:?}", char::from_digit(10,16));
                                          // Some('a')
}
```

String Slices - &str

`&str` - an immutable **borrowed** view into string bytes.

```rust
fn main() {
  // String literals have static lifetime
  let hello: &str = "Hello 🦀";

  // Byte length (not char count!)
  println!("{}", hello.len());           // 10 (🦀 = 4 bytes)

  // Char count
  println!("{}", hello.chars().count()); // 7

  // Slice (byte-indexed - must be on char boundaries!)
  let world: &str = &hello[6..10];        // 6-9 (10 excluded)
  println!("{world}");                    // 🦀

  // Iterate characters
  for c in hello.chars().take(5) {
    print!("{c} ");    // H e l l o
  }
}
```

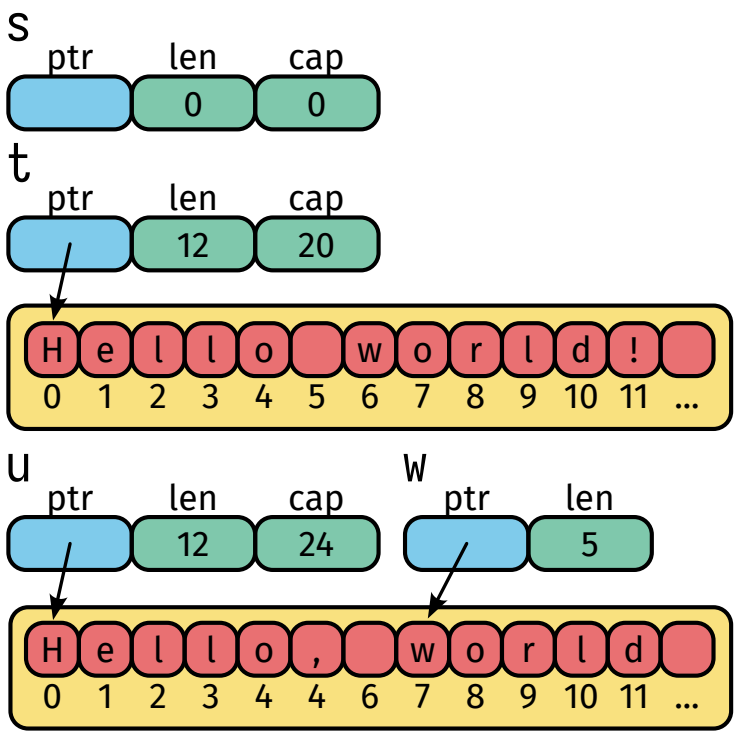
They are stored as literal data in the binary and have a `'static` lifetime.

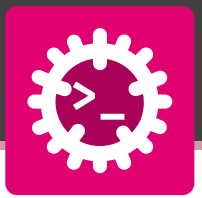


Owned Strings - String

String - heap-allocated, growable, UTF-8 text:

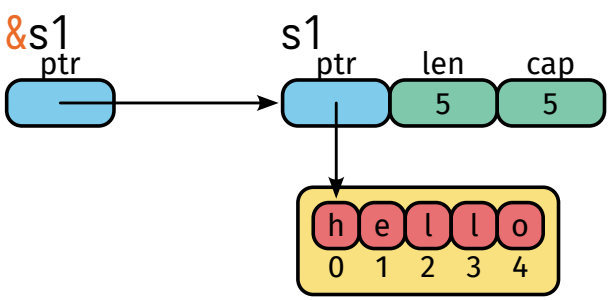
```rust
fn main() {
  let s = String::new();              // empty string

  // Append
  let mut t = String::from("Hello"); // from literal
  t.push(' ');                        // single char
  t.push_str("world!");               // string slice
  println!("{t}");                    // Hello world!
  println!("len={} cap={}", t.len(), t.capacity());

  // Insert
  let mut u = String::from("Hello world!");
  u.insert(5, ',');                   // insert char at index
  println!("{u}");                    // Hello, world!

  // Remove
  u.pop();                            // removes last char
  println!("{u}");                    // Hello, world

  // Slice
  let w = &u[7..12];                  // "world" (7-11)
}
```



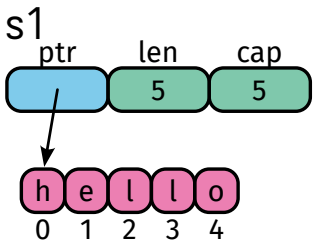


String	&str
Owned (on heap)	Borrowed (a view)
Mutable	Immutable
Has len, capacity	Has len only
Growable	Fixed slice
Allocates memory	Zero allocation

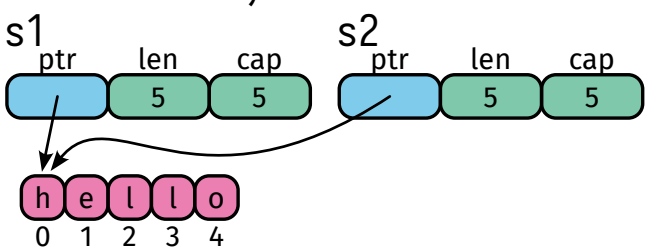
```
let s1 = String::from("hello");
&s1
```



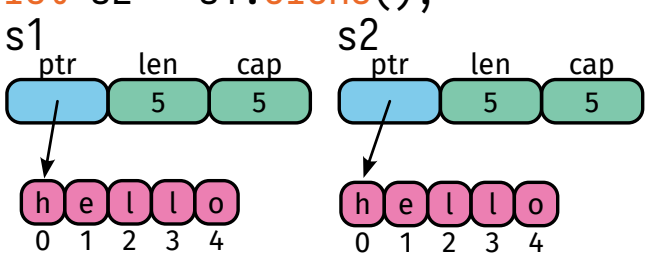

```
let s1 = "hello";
```



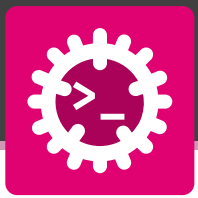

```
let s2 = s1;
```




```
let s2 = s1.clone();
```



String Operations



Searching:

```
let s = "Hello, Rust world!";

println!("{}", s.contains("Rust"));     // true
println!("{}", s.starts_with("Hello")); // true
println!("{}", s.ends_with("!"));       // true
println!("{:?}", s.find("Rust"));       // Some(7)
println!("{:?}", s.rfind('o'));         // Some(13)
```

Replacing and splitting:

```
let r = s.replace("Rust", "Go");
println!("{r}");   // Hello, Go world!

let words: Vec<&str> = s.split_whitespace().collect();
println!("{words:?}");
// ["Hello,", "Rust", "world!"]

let parts: Vec<&str> = "a,b,c".split(',').collect();
println!("{parts:?}"); // ["a", "b", "c"]
```

Trimming and transforming:

```
let padded = "  hello world  ";
println!("'{}'", padded.trim());       // 'hello world'
println!("'{}'", padded.trim_start()); // 'hello world  '
println!("'{}'", padded.trim_end());   // '  hello world'

println!("{}", "hello".to_uppercase()); // HELLO
println!("{}", "WORLD".to_lowercase()); // world

println!("{}", "=".repeat(20)); // ====================
```

Collecting:

```
// Chars to Vec
let chars: Vec<char> = "abc".chars().collect();
println!("{chars:?}"); // ['a', 'b', 'c']

// Join lines
let lines = ["line1", "line2", "line3"];
let joined = lines.join("\n");
println!("{joined}");  //line1\nline2\nline3
```

String Conversion

Value → String:

```rust
fn main() {
    let n: i32  = 42;
    let f: f64  = 3.14;
    let b: bool = true;

    // to_string() works on any Display type
    let s1 = n.to_string();   // "42"
    let s2 = f.to_string();   // "3.14"
    let s3 = b.to_string();   // "true"
    println!("{s1} {s2} {s3}");

    // format! - more control
    let msg = format!("n={n:05}, f={f:.2}");
    println!("{msg}");  // n=00042, f=3.14
}
```

String → **value** - parse():

```rust
let text = "42";
let n: i32 = text.parse().expect("bad input");
```

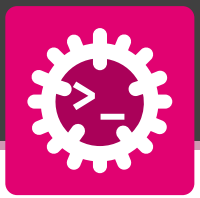
&str → String - to_owned():

```rust
let a = "hi".to_owned();      // clearest for &str
let b = "hi".to_string();     // more general
let c = String::from("hi");   // explicit
let d: String = "hi".into();  // idiomatic generics

println!("{a} {b} {c} {d}");  // hi hi hi hi
```

When to use which:

Method	Best for
to_string()	any Display type → String
to_owned()	&str specifically → String
String::from	explicit, readable constructor
.into()	idiomatic generic / trait code
.parse()	text → numeric or other types
format!(...)	multi-value or formatted strings

String Formatting println!(), print!(), format!() and write!()



Arguments

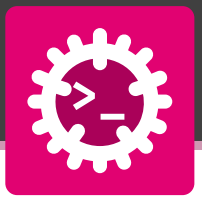
```
let s = format!("Hello, {}, the answer is {}", "Alice", 42);                  // Positional Arguments
let s = format!("Hello, {name}, the answer is {age}", name = "Bob", age = 7); // Named Arguments
let s = format!("{0} is {1} years old. {0} loves Rust!", "Carol", 25);        // Reusing Arguments
```

Align

```
format!("{:<8}", 1);  // 1.......  - left   aligned
format!("{:>8}", 1);  // .......1  -  right aligned
format!("{:^8}", 1);  // ...1....  - center aligned
```

Padding

```
format!("{:*<8}", 1); // 1*******  - left  align, padding "*"
format!("{:0>8}", 1); // 00000001  - right align, padding "0"
format!("{:>08}", 1); // 00000001  - right align, padding leading "0"
```



```
format!("{: >8}", 1); //        1 - right align, padding " "
format!("{:#^8}", 1); // ###1#### - right align, padding "#"
```

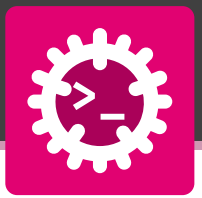
Sign

```
format!("{:>+8}", 1);   //       +1 - right align, sign +
format!("{:>8}", -1);   //       -1 - right align, sign -
format!("{:>+#08}", 1); // +0000001 - right align, sign +, padding leading "0"
format!("{:0>+#8}", 1); // 000000+1 - right align, padding "0", sign +
```

Precision

```
format!("{:*<8}", 1); // 1******* - left  align, padding "*"
format!("{:0>8}", 1); // 00000001 - right align, padding "0"
format!("{:>08}", 1); // 00000001 - right align, padding leading "0"
format!("{: >8}", 1); //        1 - right align, padding " "
format!("{:#^8}", 1); // ###1#### - right align, padding "#"
```

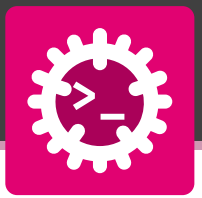
String Formatting println!(), print!(), format!() and write!()



Type

```
format!("{:}", E as u32);         // 2                    - Integer (follows type)
format!("{:}", E);                // 2.718281828459045    - Float (follows type)
format!("{:b}", 49374);           // 1100000011011110     - Binary
format!("{:#b}", 49374);          // 0b1100000011011110   - Binary alternative (adds "0b")
format!("{:#034b}", 49374);       // 0b00000000000000001100000011011110 - Binary 32bit (adds "0b" and 32 digits)
format!("{:o}", 49374);           // 140336               - Octal
format!("{:#o}", 49374);          // 0o140336             - Octal alternative (adds "0o")
format!("{:#018o}", 49374);       // 0o0000000000140336   - Octal 16 digits (adds "0o" and 16 digits)
format!("{:x}", 49374);           // c0de                 - Hex lower case
format!("{:#x}", 49374);          // 0xc0de               - Hex lower case alternative (adds "0x")
format!("{:#010x}", 49374);       // 0x0000c0de           - Hex 8 digits (adds "0x" and 8 digits)
format!("{:X}", 49374);           // C0DE                 - Hex upper case
format!("{:#X}", 49374);          // 0xC0DE               - Hex upper case alternative (adds "0X")
format!("{:#010X}", 49374);       // 0x0000C0DE           - Hex 8 digits (adds "0X" and 8 digits)
format!("{:e}", E.powf(123.0));   // 2.6195173187490456e53 - Scientific notation lower case (adds "e")
format!("{:E}", E.powf(123.0));   // 2.6195173187490456E53 - Scientific notation upper case (adds "E")
format!("{:p}", &1);              // 0x1021872c0          - Pointer address
```

Format Syntax Mini-Guide



```rust
format!(format_string, arguments...)
```
Rust

```
formatting = [argument][:][[fill]align][sign][#][0][width][.precision][type]
argument   = integer | identifier
fill       = <any character>
align      = "<" | ">" | "^"
sign       = "+"
"#"        = alternate formatting
"0"        = zero-padding
width      = integer
precision  = integer
type       = "b" | "o" | "x" | "X" | "e" | "E" | "p" | "?"
```

```
"{" [argument][":"][[fill]align][sign]["#"]["0"][width][.precision][type] "}"
       /            /   /     /   /   /                \       \
      /            /   /     /   /   |                 \       \
     /            /   /     /   /    |- padding "0"      \       |
    /            /   /     /   |                         \      |
   /            /   /      |   |- alternate formatting     \     |
  /            /   /       |   |- (human-reable)            \    |
 /            /   /        |                                 |   |
|            /   /         |- "" - default                   |   |
|           /   /          |- + - + prefix       by tschinz  |   |
|          /   |                                             |   |
|         /    |                nbr of descendants - integer -|   |
|        |     |                                              |
|        |     |- < - left (default)      "" - default follows type -|
|        |     |- > - right              "b" - binary ———————————|
|        |     |- ^ - center             "o" - octal ————————————|
|        |                               "x" - hex ——————————————|
|        |- any character for padding    "X" - Hex ——————————————|
|                                        "e" - scientific ———————|
|- "" - follows arg sequence (default)   "E" - Scientific ———————|
|- integer - selects arg by position     "p" - pointer addr ————|
|- identifier - selects arg by name      "?" - debug (trait req) ———|
```

```rust
print!("{0:*>+10.3e}", std::f64::consts::E)
```

https://wiki.zahno.dev/blog/2024/11/20/rust-string-formatting---an-ascii-art-cheatsheet.html

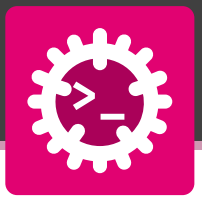
Advanced Formatting

Building a formatted table:

```
fn main() {
  let items = [
    ("Alice", 92, 98.7),
    ("Bob",   87, 91.2),
    ("Carol", 95, 99.1),
  ];
  // Header (left-align name, right-align numbers)
  println!("| {:<5} | {:>5} | {:>6} |", "Name", "Score", "Pct");
  // Alignment row (: marks alignment direction)
  println!("|:{0:-<6}|{0:-<6}:|{0:-<7}:|", "");
  // Data rows
  for (name, score, pct) in items {
    println!("| {:<5} | {:>5} | {:>5.1}% |", name, score, pct);
  }
}
```

```
| Name  | Score |    Pct |
|:------|------:|-------:|
| Alice |    92 |  98.7% |
| Bob   |    87 |  91.2% |
| Carol |    95 |  99.1% |
```

Exercise - Text Types



Part 1 - What is the output?

Trace the conversions and format specifiers.

```
fn main() {
  let mut s = String::new();
  s.push_str("Rust");
  s.push('!');

  let n: i32 = "42".parse().unwrap();
  let label = "id".to_owned() + "-" + &n.to_string();

  println!("{s} {}", s.len()); // a
  println!("{label}");         // b
  println!("[{:>6}]", "hi");   // c
  println!("[{:*<6}]", "hi");  // d
  println!("{:08.3}", 3.14159);// e
  println!("{:#x}", 255);      // f
}
```

Part 2 - Build the string

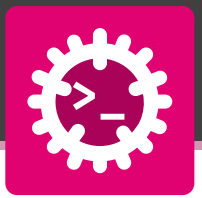
Given the bindings below, write a **single** `format!` call that produces **exactly** this line:

```
Bob   :  87
```

```
let name = "Bob"; // left-aligned, width 6
let score = 87;   // right-aligned, width 3
let line: String = format!(/* ? */);
```

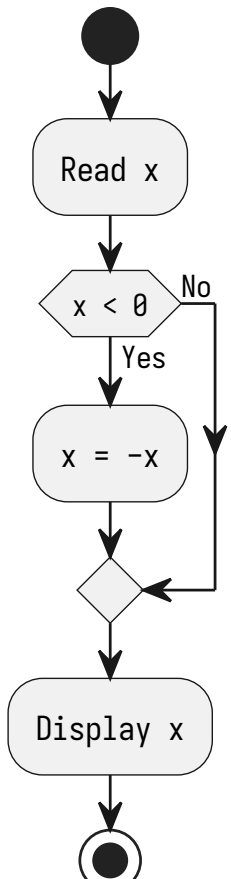
Control Flow

`if`, `else if`, `else`, `match`, `loop`s, and more



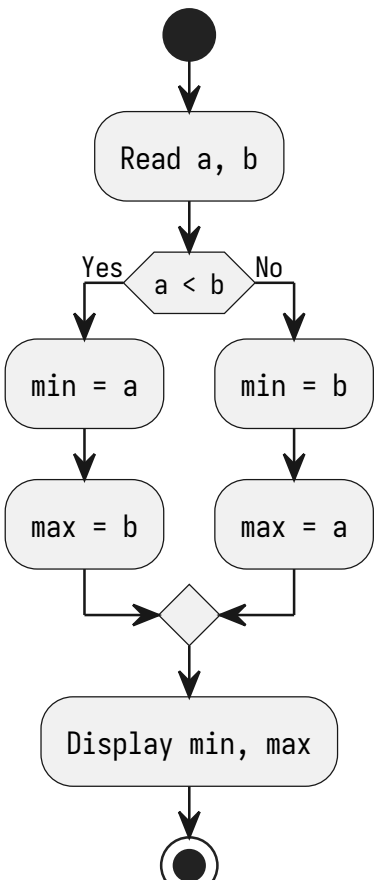
`if` is an **expression**:

```
fn main() {
  let mut x: i32 = -7;

  // Statement form
  if x < 0 {
    x = -x; // make positive
  }
  println!("{x}"); // +7

  // Expression form - both arms must return same type
  x = if x < 0 { -x };
  println!("{x}"); // +7
}
```



`if` / `else` - conditional branch; `if` is an **expression**:

```
fn main() {
  let (a, b): (i32, i32) = (42, 7);
  let (max, min);

  // Statement form
  if a < b {
    max = b; min = a;
  } else {
    max = a; min = b;
  }
  println!("Max {max}, Min {min}");

  // Expression form - both arms must return same type
  let (a, b): (i32, i32) = (7, 42);
  let (max, min);
  (max, min) = if a < b { (b, a) } else { (a, b) };
  println!("Max {max}, Min {min}");
}
```

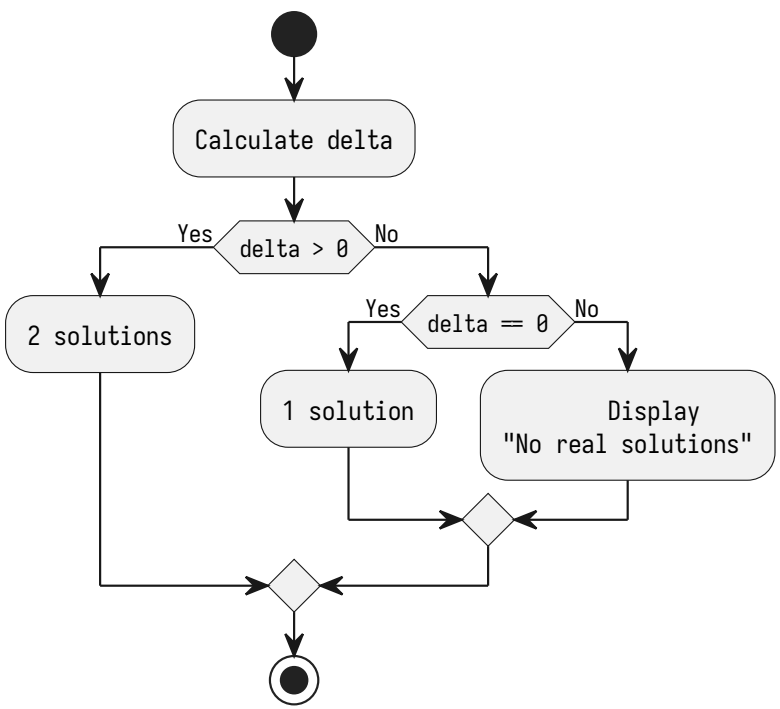


`else if` - chained conditions

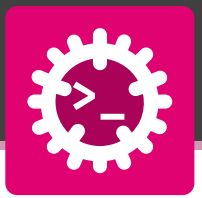
`else if` - evaluates branches top-to-bottom, first match wins:

```rust
fn main() {
  // ax² + bx + c = 0
  let (a, b, c) = (1.0, -3.0, 2.0);

  // Δ = b² - 4ac
  let delta = b * b - 4.0 * a * c;

  if delta > 0.0 {
    // x₁,₂ = (-b ± √Δ) / (2a)
    let x1 = (-b + delta.sqrt()) / (2.0 * a);
    let x2 = (-b - delta.sqrt()) / (2.0 * a);
    println!("x1 = {x1}, x2 = {x2}");
  } else if delta == 0.0 {
    // x = -b / (2a)
    let x = -b / (2.0 * a);
    println!("x = {x}");
  } else {
    println!("No real solutions");
} } }
```

$$y = ax^2 + bx + c \Rightarrow x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

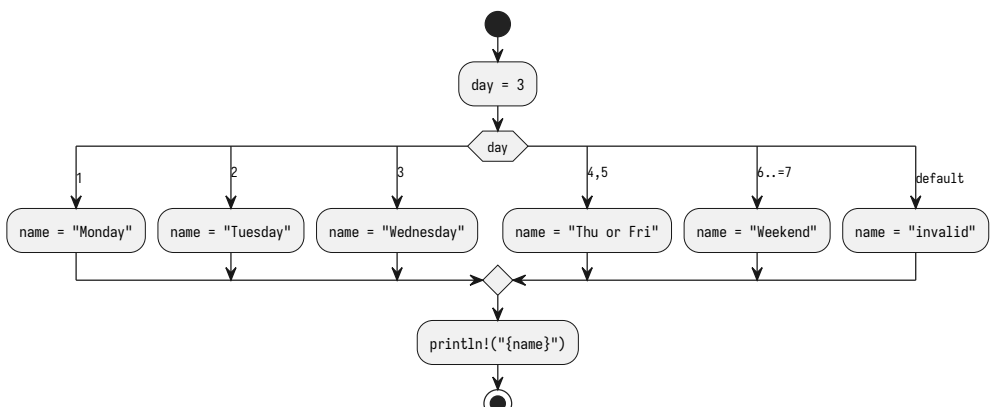


`match` - pattern matching

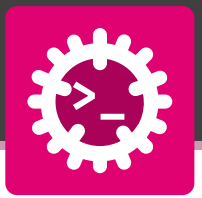


`match` - exhaustive multi-way branch on a value or pattern:

```
fn main() {
  let day: u8 = 3;
  let name = match day {
    1     ⇒ "Monday",
    2     ⇒ "Tuesday",
    3     ⇒ "Wednesday",
    4 | 5 ⇒ "Thu or Fri", // OR pattern
    6..=7 ⇒ "Weekend",    // inclusive range
    _     ⇒ "invalid",    // wildcard (must be last)
  };
  println!("{name}"); // Wednesday
}
```

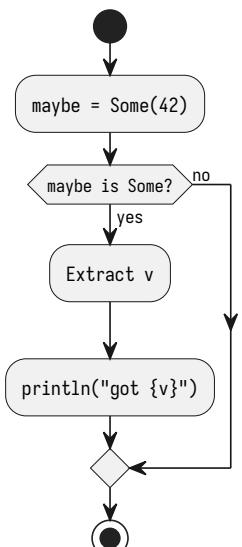


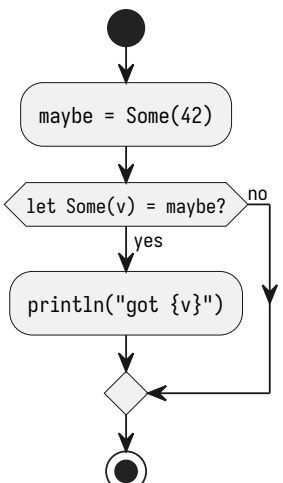
`if let` - conditional destructuring

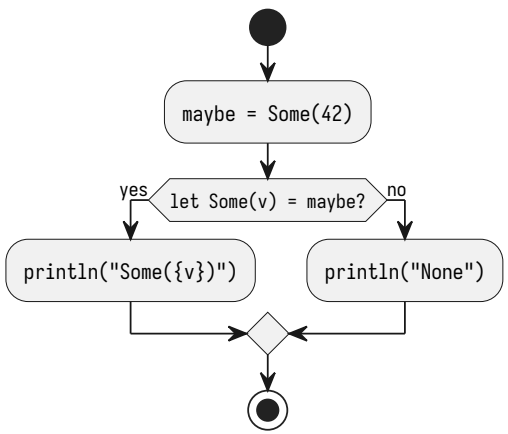


`if let` - concise pattern match for a **single** variant:

```rust
fn main() {
  let maybe: Option<i32> = Some(42);
  // Verbose: full match
  match maybe {
    Some(v) => println!("got {v}"),
    None    => {},
  }
  // Concise: if let
  if let Some(v) = maybe {
    println!("got {v}");   // 42
  }
  // with else branch
  if let Some(v) = maybe {
    println!("Some({v})"); // Some(42)
  } else {
    println!("None");
  }
}
```

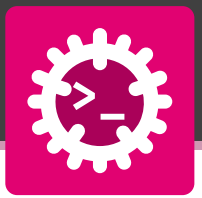






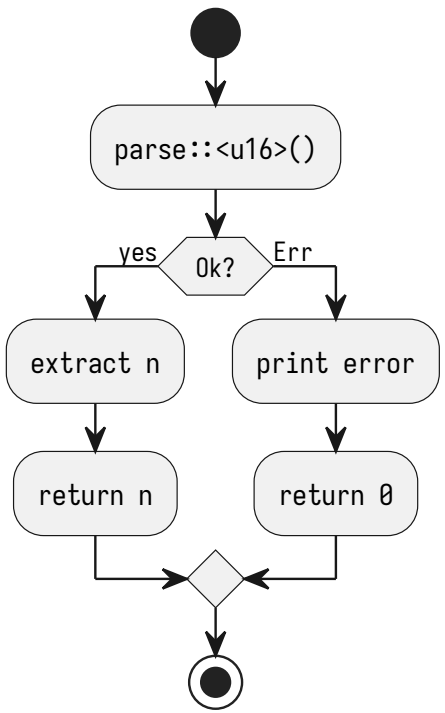
Don't worry it will get more clear with the introduction of enums later.

`let else` - bind or diverge

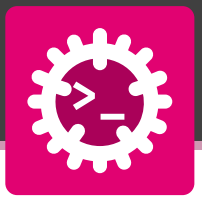


`let else` - bind a pattern or **diverge** (return / panic / break):

```
fn parse_port(s: &str) -> u16 {
  // Must diverge in the else branch
  let Ok(n) = s.parse::<u16>() else {
    eprintln!("invalid port: {s}");
    return 0;            // diverge!
  };
  // n is in scope from here on
  n
}
fn main() {
  println!("{}", parse_port("8080")); //
8080
  println!("{}", parse_port("abc"));  // 0
}
```

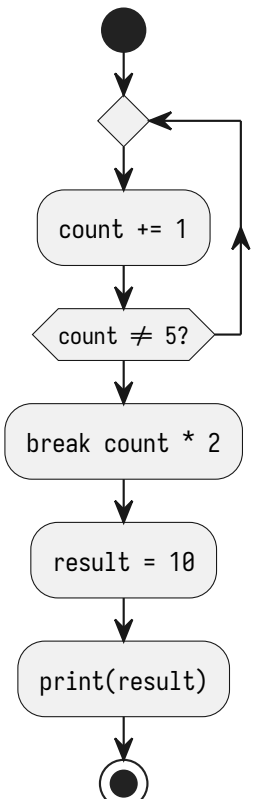


`loop` - infinite loop

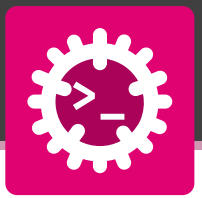


`loop` - runs forever; exit with `break`; can **return a value**:

```rust
fn main() {
  let mut count = 0;

  // loop is an expression - break carries the value
  let result = loop {
    count += 1;
    if count == 5 {
      break count * 2;   // returns 10
    }
  };
  println!("{result}"); // 10

  // Labelled loop
  'retry: loop {
    // ... break 'retry to exit by name
    break 'retry;
  }
}
```



`while` - condition-checked loop



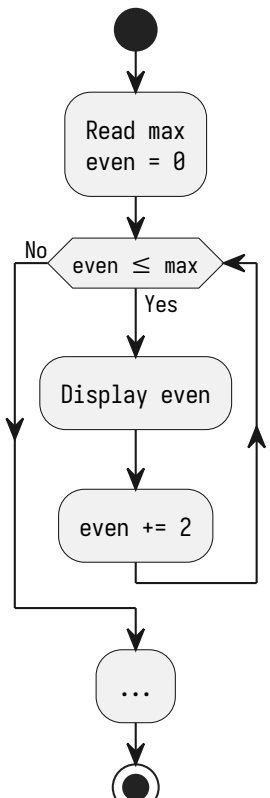
`while` - checks condition **before** each iteration:

Classic `while`

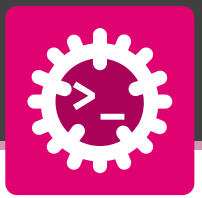
```rust
fn main() {
  let max = 10;
  let mut even = 0;
  while even ≤ max {
    println!("{even}");
    even += 2;
} }
```

`while let` - loop while pattern matches

```rust
fn main() {
  let mut stack = vec![3, 2, 1];
  while let Some(top) = stack.pop() {
    print!("{top} "); // 1 2 3
} }
```

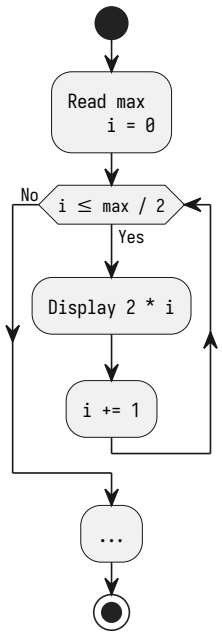


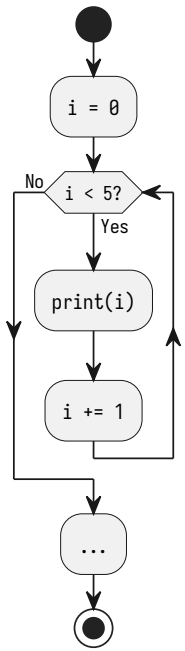
for - iterator loop

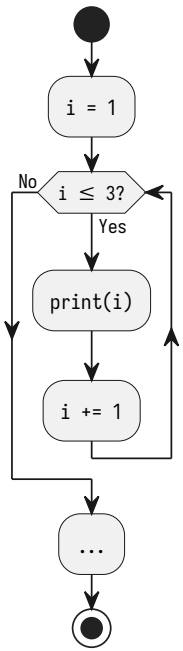


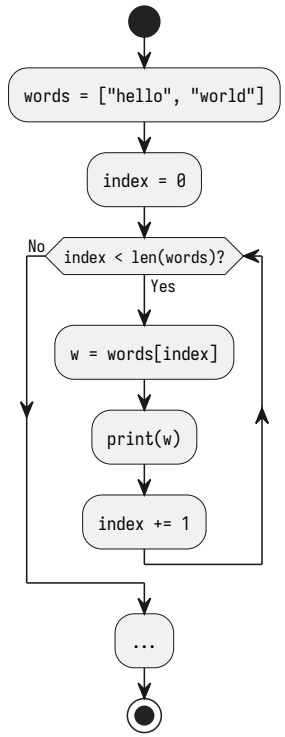
for - iterates over any IntoIterator

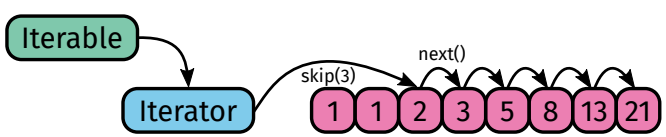
```rust
fn main() {
  let max = 10;
  for i in 0..=max / 2 {
      println!("{}", 2 * i);
  }

  // Half-open range [0, 4]
  for i in 0..5 {
    print!("{i} ");      // 0 1 2 3 4
  }
  // Inclusive range [1, 3]
  for i in 1..=3 {
    print!("{i} ");      // 1 2 3
  }
  // Iterate over a collection
  let words = ["hello", "world"];
  for w in words {
    println!("{w}");
} }
```



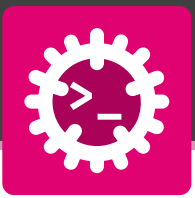








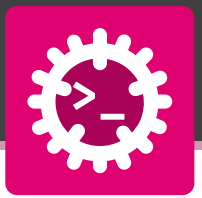
for - handy iterator adapters



```rust
fn main() {
  let words = ["alpha", "beta", "gamma"];
  let nums  = [10, 20, 30];

  // enumerate() - (index, value)
  for (i, w) in words.iter().enumerate() {
    println!("{i}: {w}"); // 0: alpha ...
  }

  // zip() - pair two iterators
  for (w, n) in words.iter().zip(nums) {
    println!("{w}={n}");  // alpha=10 ...
  }

  // rev() - reverse a range
  for i in (1..=5).rev() {
    print!("{i} ");       // 5 4 3 2 1
  }

  // step_by() - custom step (replaces i+=3)
  for i in (0..12).step_by(3) {
    print!("{i} ");       // 0 3 6 9
  }
}
```

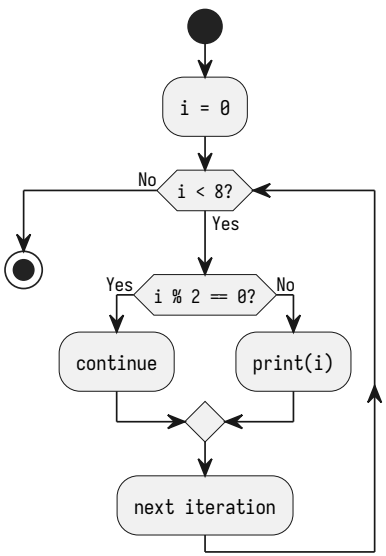
```rust
fn main() {
  let nums = [1, 2, 3, 4, 5, 6];

  // filter() - skip elements
  for x in nums.iter().filter(|&&x| x % 2 == 0) {
    print!("{x} ");       // 2 4 6
  }

  // map() - transform each element
  for x in nums.iter().map(|&x| x * x) {
    print!("{x} ");       // 1 4 9 16 25 36
  }

  // chain() - join two iterables
  let a = [1, 2];
  let b = [3, 4];
  for x in a.iter().chain(b.iter()) {
    print!("{x} ");       // 1 2 3 4
  }

  // flatten() - one level of nesting
  let nested = [[1, 2], [3, 4]];
  for x in nested.iter().flatten() {
    print!("{x} ");       // 1 2 3 4
} }
```

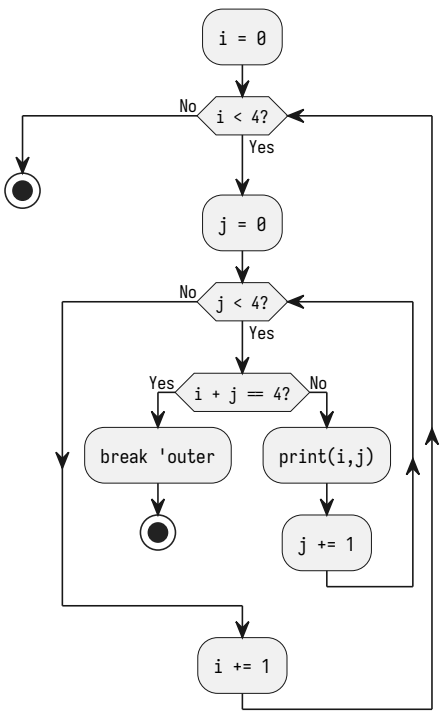
break / continue - loop control



`continue` skips the rest of the body; `break` exits the loop:

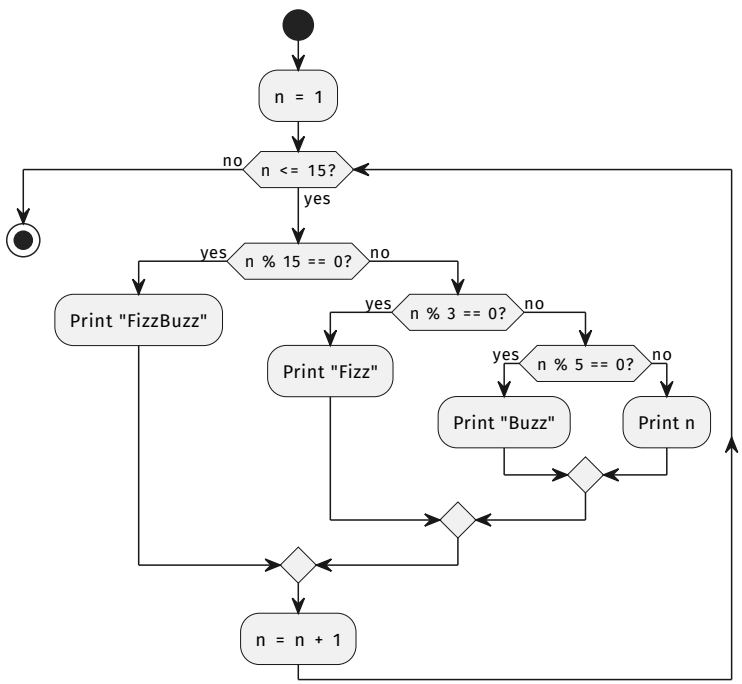
```
fn main() {
  // continue: skip even numbers
  for i in 0..8 {
    if i % 2 == 0 { continue; }
    print!("{i} ");       // 1 3 5 7
  }

  // break with label: exit outer loop
  'outer: for i in 0..4 {
    for j in 0..4 {
      if i + j == 4 { break 'outer; }
      print!("{i},{j} ");
    }
  }
}
```





Exercise - Control Flow

Translate the flowchart below into Rust using a `for` loop and an `if` / `else if` / `else` chain.



Order matters: the **most specific** test (`% 15`) must come **first**, otherwise `% 3` would catch it.

Bonus: can you express the same logic with a single `match`?

Functions

Let’s make spaghetti!

Function Anatomy

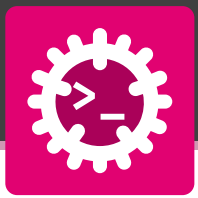
`fn` - explicit type on every parameter and on the return:

```rust
//         parameters (name: Type)
//         |       |       ┌─ return type
fn add(a: i32, b: i32) -> i32 {
  a + b     // implicit return: last expr, no `;`
}

// no return type `-> T` means `-> ()` (unit)
fn greet(name: &str) {
  println!("Hello, {name}!");
}

// explicit early return
fn abs(x: i32) -> i32 {
  if x < 0 { return -x; }  // early exit
  x                        // implicit return
}

fn main() {
  println!("{}", add(3, 4)); // 7
  greet("Alice");            // Hello, Alice!
  println!("{}", abs(-9));   // 9
}
```

teaser: this is where functions can go

```rust
//     ┌──────────────────────────────────── function name
//     │     ┌──────────────────────────────── lifetime parameter
'a
//     │     │     ┌──────────────────────────── generic type T
//     │     │     │     ┌──────────────────────── first input slice
//     │     │     │     │     ┌──────────────────── second input slice
//     │     │     │     │     │                 ┌─ return type:
//     │     │     │     │     │                 │  a borrowed
//     │     │     │     │     │                 │  slice
fn longest<'a, T>(x: &'a [T], y: &'a [T]) -> &'a [T]
where T: Ord,
//    └─ T must be orderable, so slices [T] can be compared
//└──── trait bound
{
  if x > y { x } else { y }
}
```

Return Values



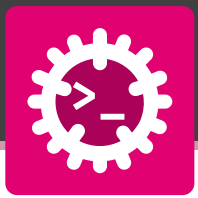
Single value and **no value** (()):

```rust
fn square(x: f64) -> f64 { x * x }

// returns () implicitly - same as void
fn log(msg: &str) {
  println!("[LOG] {msg}");
}

// early return to guard against bad input
fn safe_div(a: f64, b: f64) -> f64 {
  if b == 0.0 { return f64::NAN; }
  a / b
}

fn main() {
  println!("{}", square(4.0));        // 16
  log("started");
  println!("{}", safe_div(1.0, 0.0)); // NaN
}
```

Multiple values via tuple destructuring:

```rust
// return a tuple - idiomatic multi-value return
fn min_max(v: &[i32]) -> (i32, i32) {
  let (mut lo, mut hi) = (v[0], v[0]);
  for &x in v {
    if x < lo { lo = x; }
    if x > hi { hi = x; }
  }
  (lo, hi)        // return tuple
}

fn main() {
  let data = [3, 1, 4, 1, 5, 9, 2, 6];

  // destructure the tuple on the left
  let (lo, hi) = min_max(&data);
  println!("min={lo}, max={hi}"); // min=1, max=9

  // ignore one field with _
  let (_, hi) = min_max(&data);
  println!("max={hi}");           // max=9
}
```

Ownership & Borrowing



Ownership rules

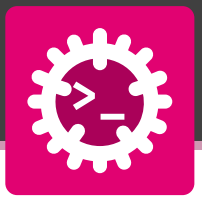
1. Each value in Rust has an **owner**.
2. There can only be **one owner** at a time.
3. When the owner goes out of scope, the value will be **dropped**.

Rules of References

1. You can have **either** one mutable reference **or** any number of immutable references.
2. References must always be valid (no dangling references).

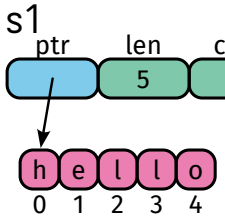
```rust
let s1 = String::from("hello");

// Ownership moves to s2
let s2 = s1;
// println!("{s1}"); // (!) error: moved

// Borrowing: many immutable refs OK
let r1 = &s2;
let r2 = &s2;
println!("{r1} {r2}");

// Or one mutable ref...
let mut s3 = String::from("world");
let m = &mut s3;
m.push_str("!");
// let r3 = &s3; // (!) error: already
// borrowed as mutable
println!("{m}");
```

Move vs Copy

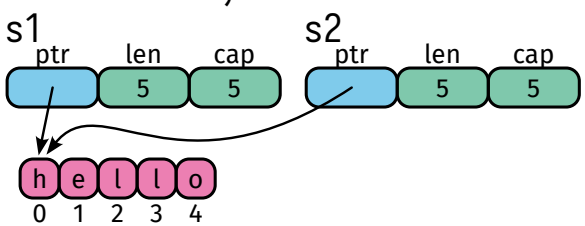


`&str` is a `Copy` type (a pointer to a string slice), while `String` is a `Move` type (a heap-allocated string):

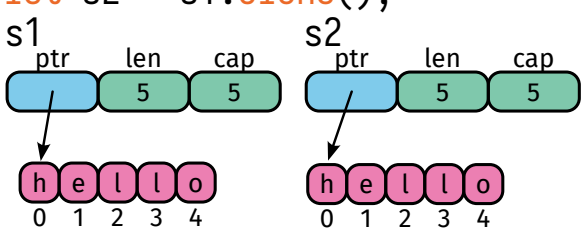
```
let s1 = "hello";
```



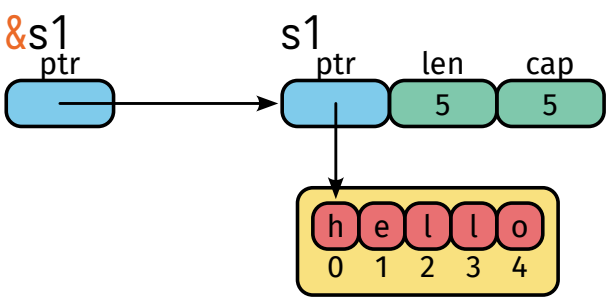

```
let s2 = s1;
```




```
let s2 = s1.clone();
```




```
let s1 = String::from("hello");
&s1
```



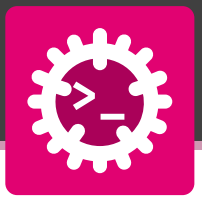
Passing by Value - Move & Copy

Passing a value **transfers** or **copies** it depending on the type:

```rust
// i32 is Copy - the caller keeps its copy
fn double(n: i32) -> i32 { n * 2 }

// String is NOT Copy - ownership moves into the fn
fn shout(s: String) -> String {
  s.to_uppercase()  // takes ownership, returns new value
}
fn main() {
  // Copy types: original still valid
  let x = 5;
  println!("{}", double(x)); // 10
  println!("{x}");           // 5 ✓ still usable

  // Move types: original is gone after the call
  let name = String::from("alice");
  let loud = shout(name);    // name moved into shout()
  // println!("{name}");     // ✗ compile error: moved!
  println!("{loud}");        // ALICE
}
```

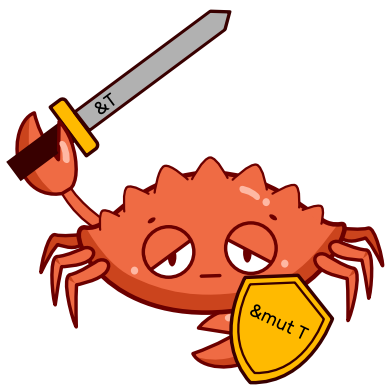
Copy **types** - value is duplicated	Move **types** - ownership transfers
bool, char	String
i8...i128, isize	Vec<T>
u8...u128, usize	HashMap<K,V>
f32, f64	Box<T>
(T, U) if all Copy	File, TcpStream
[T; N] if T: Copy	any struct by default
&T (shared ref)	&mut T (unique ref)

References - `&T` and `&mut T`

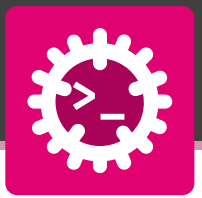


Borrow a value instead of taking ownership:

```rust
// &T  - immutable borrow: read-only, never moves
fn length(s: &str) -> usize {
  s.len()           // caller keeps ownership
}

// &mut T - mutable borrow: read+write, still no move
fn double_all(v: &mut Vec<i32>) {
  for x in v.iter_mut() { *x *= 2; }
}

fn main() {
  let hello = String::from("hello");
  println!("{}", length(&hello)); // 5
  println!("{hello}");           // ✓ still owned here

  let mut nums = vec![1, 2, 3];
  double_all(&mut nums);         // lend mutably
  println!("{:?}", nums);        // [2, 4, 6]
}
```



Syntax	Alias	Rule
T	move/copy	1 owner
&T	shared ref	many readers
&mut T	unique ref	1 writer



Anonymous functions

A closure is a function-like value written directly where it is needed.

```rust
fn main() {
  let square = |x: i32| x * x;
  println!("{}", square(4)); // 16
  let factor = 10;
  let scale = |x: i32| x * factor;
  println!("{}", scale(3)); // 30
}
```

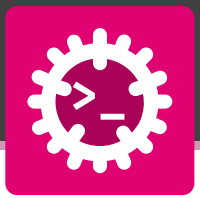
Closure syntax

```rust
|x| x + 1
|x: i32| x * x
|x: i32| -> i32 {
  x + 1
}
```

Key idea

- Anonymous function
- Can be stored in a variable
- Can use values from the surrounding scope

Exercise - Functions



Write **two small functions** for an exam grading system.

1. `swiss_grade(points: u32) -> f64`
   The exam has **5 exercises** of **0-10 points** each (`max = 50`). Convert the points to a **Swiss grade** (1-6):

$$\text{grade} = \frac{5}{\text{points}_{\text{max}}} * \text{points}_{\text{student}} + 1 \qquad (1)$$

2. `bologna_grade(grade: f64) -> char`
   Map the Swiss grade to a **Bologna letter** (see table).
3. In `main`, print points, Swiss grade (2 decimals) and the letter for `50, 45, 40, 35, 30, 25` points.

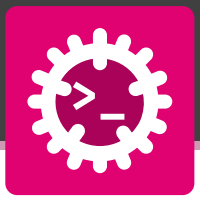
Letter	Swiss grade
A	5.75 <= 6.00
B	5.25 <= 5.74
C	4.75 <= 5.24
D	4.25 <= 4.74
E	3.75 <= 4.24
F	1.00 <= 3.74

Use as `f64` to mix u32 points with floating-point maths.

User-Defined Types

Let’s make it objects!

Enum



One type, several variants

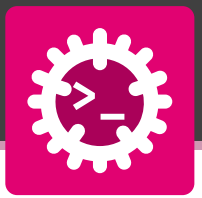
An `enum` defines a type that can be **one of several alternatives**.

```
enum TrafficLight {
  Red,
  Yellow,
  Green,
}

fn main() {
  let light = TrafficLight::Green;

  match light {
    TrafficLight::Red    => println!("stop"),
    TrafficLight::Yellow => println!("prepare"),
    TrafficLight::Green  => println!("go"),
  }
}
```

Key idea

- A value has exactly **one** variant
- Variants belong to the enum type
- `match` is used to handle all cases
- The compiler checks completeness

Enum's with Data



A variant can store values

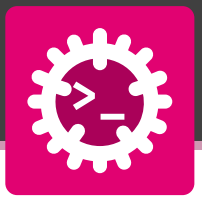
```
enum SensorValue {
  Temperature(f64),
  Pressure(f64),
  Humidity(f64),
}

fn main() {
  let value = SensorValue::Temperature(23.7);

  match value {
    SensorValue::Temperature(t) => println!("{t} °C"),
    SensorValue::Pressure(p)    => println!("{p} Pa"),
    SensorValue::Humidity(h)    => println!("{h} %"),
  }
}
```

Pattern matching extracts the value

```
SensorValue::Temperature(t)
                         ^
                         |
                  extracted value
```

Option<T> and Result<T,E> are simply enums whose variants also contain data.

Option Enum



Value or no value

`Option<T>` is used when a value may be absent.

```
fn first(values: &[i32]) -> Option<i32> {
  if values.len() > 0 {
    Some(values[0])
  } else {
    None
  }
}

fn main() {
  let data = [10, 20, 30];

  match first(&data) {
    Some(v) => println!("first = {v}"),
    None    => println!("empty list"),
  }
}
```

```
enum Option<T> {
  Some(T),
  None,
}
```

Meaning

- `Some(T)` contains a value
- `None` means no value
- Rust has no `null`

Result Enum

Success or error

`Result<T, E>` is used when an operation can fail.

```rust
fn parse_port(s: &str) -> Result<u16, String> {
  match s.parse::<u16>() {
    Ok(port) => Ok(port),
    Err(_)   => Err(String::from("invalid port")),
  }
}

fn main() {
  let result = parse_port("8080");

  match result {
    Ok(port) => println!("port = {port}"),
    Err(msg) => println!("error: {msg}"),
  }
}
```


```rust
enum Result<T, E> {
  Ok(T),
  Err(E),
}
```

Meaning

- `Ok(T)` contains the successful value
- `Err(E)` contains the error value
- The caller must handle both cases

Extracting Values from Option and Result with `unwrap()`

Sometimes we are sure that an `Option` contains `Some(T)` or a `Result` contains `Ok(T)`. `unwrap()` extracts the value directly.

`Option<T>`

```rust
fn main() {
  let port: Option<u16> = Some(8080);
  let value = port.unwrap();
  println!("port = {value}"); // 8080
}
```

```rust
fn main() {
  let port: Option<u16> = None;
  let value = port.unwrap(); // panic
  println!("port = {value}");
}
```

`Result<T, E>`

```rust
fn main() {
  let port: Result<u16, &str> = Ok(8080);
  let value = port.unwrap();
  println!("port = {value}"); // 8080
}
```

```rust
fn main() {
  let port: Result<u16, &str> = Err("invalid port");
  let value = port.unwrap(); // panic
  println!("port = {value}");
}
```

Extracting Values from Option and Result with `expect()`

> expect("message") works like unwrap(), but adds your own panic message.

```rust
fn main() {
  let port: u16 = "8080".parse::<u16>().expect("port must be a number between 0 and 65535");
}
fn main() {
  let port: u16 = "8080"
    .parse::<u16>()
    .expect("port must be a number between 0 and 65535");
}
```

Be Careful with `unwrap()` and `expect()`

Useful in examples, dangerous in real applications

`unwrap()` and `expect()` stop the program when the value is `None` or `Err`.

Acceptable in small examples or tests

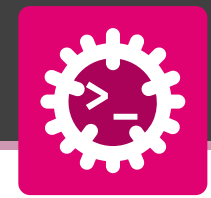
```rust
#[test]
fn parses_valid_port() {
  let port: u16 = "8080".parse::<u16>().unwrap();
  assert_eq!(port, 8080);
}
```

Better error message with `expect()`

```rust
fn main() {
  let config = std::fs::read_to_string("config.toml")
    .expect("config.toml must exist next to the program");
  println!("{config}");
}
```

⚠ Use `expect()` only when crashing is acceptable and the message helps debugging.

Be Careful with unwrap() and expect()



Better: handle the error

```
fn main() {
    let text: &str = "42";

    // unwrap: give me the value, or PANIC
    // parse() returns Result<u32, ParseIntError>
    let n: u32 = text.parse().unwrap();
    println!("{n}");           // 42

    // expect: same, but with a message
    let m: u32 = text.parse().expect("not a
number");
    println!("{m}");           // 42

    // if text were "abc", BOTH would crash:
    // thread 'main' panicked: ParseIntError ...
  }
```

Better: provide a safe fallback

```
fn main() {
  let text = "abc";
  // match: handle BOTH outcomes explicitly
  match text.parse::<u32>() {
    Ok(n)  => println!("got {n}"),
    Err(_) => println!("not a number"),
  }

  // if let: act ONLY on success, skip the rest
  if let Ok(n) = text.parse::<u32>() {
    println!("got {n}");
  }

  // unwrap_or: supply a fallback, no crash
  let n = text.parse::<u32>().unwrap_or(0);
  println!("n = {n}");       // n = 0
}
```

Be Careful with `unwrap()` and `expect()`

Method	For	Behaviour
`unwrap()`	Option/Result	extract value or panic
`expect(msg)`	Option/Result	panic with custom message
`unwrap_or(default)`	Option/Result	extract value or use default
`unwrap_or_else(f)`	Option/Result	extract value or call closure
`unwrap_or_default()`	Option/Result	extract value or Default::default()
`ok_or(err)`	Option	convert to Result with error
`ok_or_else(f)`	Option	convert to Result with closure error
`unwrap_err()`	Result	extract error or panic
`expect_err(msg)`	Result	panic with msg if not Err
`match`	Option/Result	Handle success and failure explicitly
`?`	Option/Result	Return the error to the caller

⚠ `unwrap()` is convenient for examples and quick prototypes, but can crash the program if the value is None. Use it with caution.

Named fields

A struct groups related values into a single custom type.

```rust
#[derive(Debug)]
struct Sensor {
    id: u32,
    value: f64,
    unit: String,
}

fn main() {
    let s = Sensor {
        id: 1,
        value: 23.7,
        unit: String::from("°C"),
    };

    println!("{:?}", s);
    println!("Sensor {}: {} {}", s.id, s.value, s.unit);
}
```

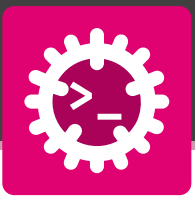
```
Sensor { id: 1, value: 23.7, unit: "°C" }
Sensor 1: 23.7 °C
```

Key idea

- Create your own type
- Group related data together
- Fields have names and types
- Access fields with `.`
- Improves readability over tuples

```
sensor.id
sensor.value
sensor.unit
```

Exercise - Option and Result



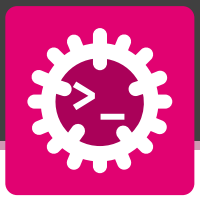
Define a small **struct** and write one `Option` and one `Result` function:

1. Struct `Item { name: String, stock: u32 }`.
2. `take(item: &Item) -> Option<u32>` - `Some(stock - 1)` if `stock > 0`, otherwise `None`.
3. `parse_qty(s: &str) -> Result<u32, String>` - `Ok(n)` if `s` parses as a `u32`, otherwise `Err("not a number")`.
4. In `main`, create an `Item` and print the result of each function.

Reach for `Option` when a value **might be missing**, and `Result` when it **might fail**.

Ownership and Borrowing

And what the hell are Lifetimes `'a`?



Every value has one owner

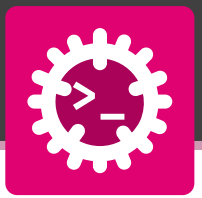
In Rust, every value has an **owner**. When the owner goes out of scope, the value is dropped.

```
fn main() {
  {
    let name = String::from("Ferris");
    println!("{name}");
  } // name goes out of scope here
  println!("{name}"); // error
}
```

Ownership rules

1. Each value in Rust has exactly one owner.
2. There can only be one owner of a value at any given time.
3. When the owner goes out of scope, the value is dropped

Ownership is Rust's way to manage memory safely at compile time.



Ownership can be transferred

Values like `String` are moved when assigned to another variable.

```rust
fn main() {
  let a = String::from("Ferris");
  let b = a; // ownership moves to b
  println!("{b}");
  println!("{a}"); // error
}
```

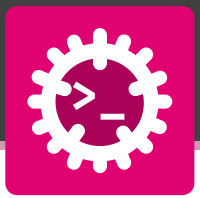
What happens?

```rust
let b = a;
```

- a no longer owns the string
- b is now the owner
- The data is not copied
- This prevents double-free errors

After a move, the old variable can no longer be used.

Ownership - Copy



Some values are copied instead of moved

Simple values like integers, floats, booleans, and characters are copied.

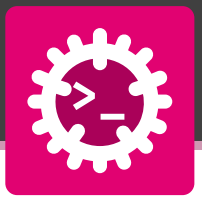
```rust
fn main() {
    let a = 42_u8;
    let b = a; // copy
    println!("a = {a}");
    println!("b = {b}");
}
```

```rust
fn main() {
  let p = (10_u8, 20_u128);
  let q = p;
  println!("{:?}", p);
  println!("{:?}", q);
}
```

Copy types

- i32, u32, f64
- bool
- char
- Tuples containing only Copy types

Copy values are small and simple. Rust duplicates them automatically.



Function calls can move ownership

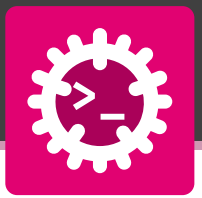
Passing a non-Copy value to a function transfers ownership.

```
fn print_name(name: String) {
    println!("{name}");
}
fn main() {
    let name = String::from("Ferris");
    print_name(name);
    println!("{name}"); // error
}
```

Why?

- name is moved into print_name
- The function becomes the owner
- At the end of the function, the value is dropped

If a function does not need ownership, pass a reference instead.



Read without taking ownership

A reference lets a function use a value without owning it.

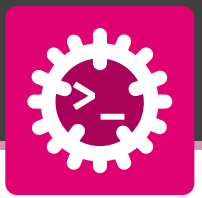
```rust
fn print_name(name: &String) {
    println!("{name}");
}
fn main() {
    let name = String::from("Ferris");
    print_name(&name);
    println!("{name}"); // still valid
}
```

Borrowing

- `&String` is a reference
- The function can read the value
- Ownership stays in `main`
- The value is not dropped by the function

```rust
print_name(&name);
//         ^
//         borrow
```

Ownership - Mutable Borrowing



Change without taking ownership

A mutable reference allows a function to modify a value.

```rust
fn add_unit(text: &mut String) {
    text.push_str(" °C");
}
fn main() {
    let mut value = String::from("23.7");
    add_unit(&mut value);
    println!("{value}");
}
```

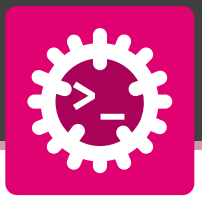
Rules

- The variable must be declared `mut`
- The reference must be `&mut`
- Only one mutable reference is allowed at a time

```rust
let mut value = String::from("23.7");
add_unit(&mut value);
```

Mutable borrowing allows controlled modification without losing ownership.

Ownership - Borrowing Rules



Many readers or one writer

This prevents data races and invalid references at compile time.

Allowed: many immutable borrows

```rust
fn main() {
    let value = String::from("23.7 °C");
    let a = &value;
    let b = &value;
    println!("{a}");
    println!("{b}");
}
```

Not allowed: read and write at same time

```rust
fn main() {
    let mut value = String::from("23.7");
    let read = &value;
    let write = &mut value; // error
    println!("{read}");
    println!("{write}");
}
```


Rule of thumb: either many readers or exactly one writer.

The compiler tracks how long references are valid

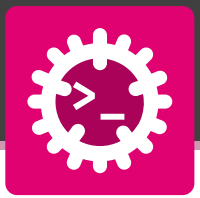
A **lifetime** is the span for which a reference stays valid. The **borrow checker** rejects any reference that could outlive the data it points to - so no dangling pointers and no use-after-free, all without a garbage collector.

✗ Dangling reference - rejected at compile time

```
fn main() {
  let r;
  {
    let value = String::from("Ferris");
    r = &value;       // r borrows value
  }                   // value dropped here!
  println!("{r}");    // r would read freed memory
}
```

```
error[E0597]: `value` does not live long enough
  r = &value;
      ^^^^^^ borrowed value does not live long
enough
  }   - `value` dropped here while still borrowed
  println!("{r}");
             ─── borrow later used here
```

Lifetimes



✓ Valid - the reference never outlives its owner

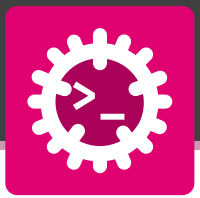
```
fn main() {
  let value = String::from("Ferris"); //  ┐ value
  let r = &value;                     //  ┌┐ r
  println!("{r}");                    //  ┌┘
}                                     //  └┘ both end
```

Key idea

- A reference must **not outlive** the value it borrows
- Checked at **compile time only** - zero runtime cost
- Usually **inferred**; explicit `'a` syntax shown next

> Lifetimes don’t change **when** a value is dropped - they let the compiler **prove** every reference is valid for as long as it is used.

Lifetime Annotations 'a



Naming a lifetime so the compiler can relate references

Lifetimes are usually **inferred**. You only name one when a function **returns a reference** and the compiler cannot tell which input it borrows from.

```rust
// 'a links the inputs to the output:
// "the result lives as long as both inputs do"
fn longest<'a>(in1: &'a str, in2: &'a str) -> &'a str {
    if in1.len() > in2.len() { in1 } else { in2 }
}

fn main() {
    let s1 = String::from("Ferris");
    let s2 = String::from("Rust");
    println!("{}", longest(&s1, &s2)); // Ferris
}
```

Reading the signature

- <'a> - declare a lifetime named a
- &'a str - “a reference valid for 'a”
- inputs and output **share** 'a, so the returned reference cannot outlive in1 or in2

Think of 'a **as** a **label** that links references together

Rule of thumb: if a function takes references and returns one, it usually needs a lifetime to say **which** input the result borrows from.

Exercise - Ownership and Borrowing

Each snippet below **does not compile**. **Find the ownership / borrowing problem** and say how you would fix it.

A

```
let s = String::from("hi");
let t = s;
println!("{s} and {t}");
```

B

```
let mut v = vec![1, 2, 3];
let first = &v[0];
v.push(4);
println!("first = {first}");
```

C

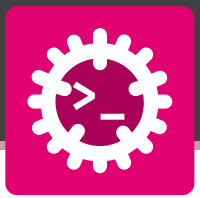
```
fn first_word() -> &str {
  let s = String::from("hello world");
  s.split(' ').next().unwrap()
}
```

Think about **who owns** the value, **how long** a reference stays valid, and the rule **many readers XOR one writer**.

Composite Types

Let’s mix and match!

Composite Types



Grouping several values - Composite types store several values together.

Type	Use case
`[T; N]`	Fixed-size array
`Vec<T>`	Growable list
`HashMap<K, V>`	Fast key-value lookup
`BTreeMap<K, V>`	Sorted key-value lookup
`HashSet<T>`	Unique values, fast lookup
`BTreeSet<T>`	Unique values, sorted
`VecDeque<T>`	First In First Out (FIFO) queue

Most collection types are generic i.e. support any type.

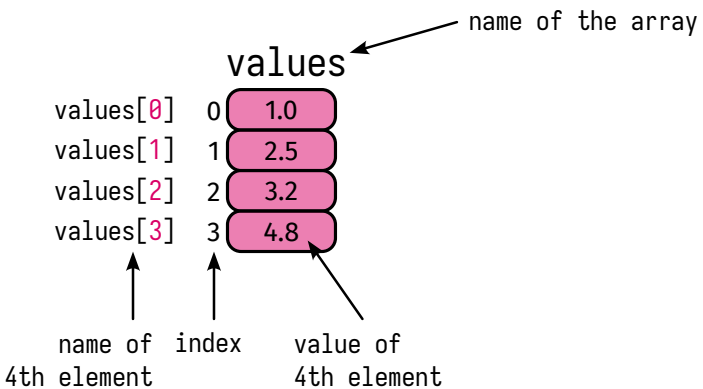
```rust
use std::collections::{
    HashMap,
    HashSet,
    BTreeMap,
    BTreeSet,
    VecDeque,
};
fn main() {
    let numbers = [1, 2, 3];           // array
    let names = vec!["Ana", "Bob"];    // vector

    let mut ages = HashMap::new();     // HashMap
    ages.insert("Ana", 21);

    let mut queue = VecDeque::new();   // VecDeque
    queue.push_back("task 1");
    queue.push_back("task 2");
}
```

Arrays `[T; N]`

`[T; N]` Fixed size, same type - An array stores a fixed number of values of the same type.

```rust
fn main() {
    let values: [f32; 4] = [1.0, 2.5, 3.2, 4.8];

    println!("length = {}", values.len()); // 4
    println!("first  = {}", values[0]);    // 1
    println!("last   = {}", values[3]);    // 4.8

    for value in values {
        println!("{value}"); // 1
    }                        // 2.5
}                            // 3.2
                             // 4.8
```

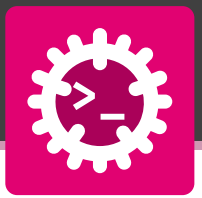
```rust
[i32; 4]
  │   └─ length
  └──── element type
```



Array properties

- Fixed length
- Length is part of the type
- Stored contiguously
- Fast indexed access
- Useful for small fixed data

Multidimensional Arrays `[[T; N]; M]`

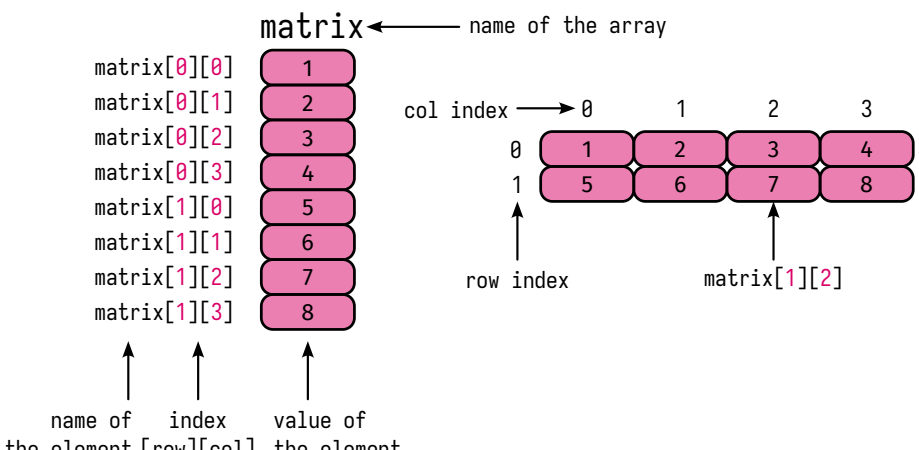


`[[T; N]; M]` Arrays can contain arrays - A multidimensional array is an array whose elements are also arrays.

```rust
fn main() {
  let image: [[u8; 3]; 2] = [
    [255, 128, 0],
    [10,  20, 30],
  ];
  println!("{}", image[0][0]); // 255
  println!("{}", image[1][2]); // 30

  for row in image {
    for pixel in row {
      print!("{pixel:3} "); // 255 128  0
    }                       //  10  20 30
    println!();
} }
```

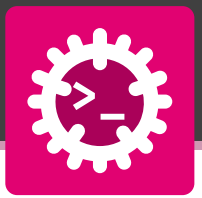
```rust
[[u8; 3]; 2]
  │    │  └─ rows
  │    └──── columns
  └─────────── element type
```

```rust
let matrix: [[i32; 4]; 2] = [[1, 2, 3, 4], [5, 6, 7, 8]];
```



Typical use cases

- Small matrices
- Lookup tables
- Image kernels
- Fixed sensor grids

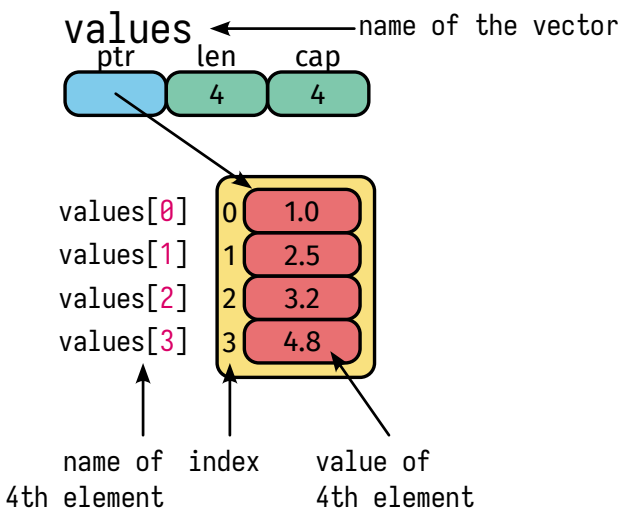


Vec<T> **Growable list** - A vector stores values of the same type and can grow at runtime.

```rust
fn main() {
  let mut readings: Vec<f64> = Vec::new();

  readings.push(23.7);
  readings.push(24.1);
  readings.push(23.9);

  println!("len = {}", readings.len());
  println!("first = {}", readings[0]);

  for value in &readings {
    println!("{value} °C");
  }
}
```

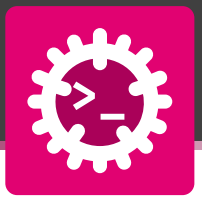
```rust
let mut values = Vec::new();
values.push(10);
values.push(20);
```

```rust
let values: Vec<f32> = vec![1.0, 2.5, 3.2, 4.8]
```



Vector properties

- Dynamic length
- Same element type
- Fast access by index
- Can add values with push



Safe Access with `get()`, avoiding index panics.

Indexing with `[i]` panics if the index is invalid. `get(i)` returns an `Option`.

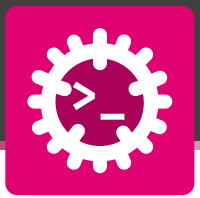
```rust
fn main() {
  let readings = vec![23.7, 24.1, 23.9];

  // Direct indexing
  println!("{}", readings[1]); // 24.1

  // Safe access
  match readings.get(10) {
    Some(value) => println!("{value}"),
    None => println!("no such reading"),
  }
}
```

Why `Option`?

```rust
let value: Option<&f64> = readings.get(10)
```

- `Some(value)` if the index exists
- `None` if the index is invalid

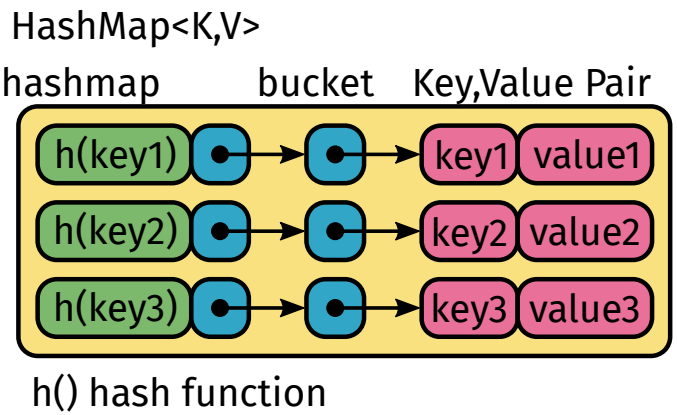
Prefer `get()` when the index may come from user input, files, sensors, or network data.



HashMap<K, V> **Fast key-value lookup** - Stores values under keys.

```rust
use std::collections::HashMap;

fn main() {
  let mut sensors = HashMap::new();
  sensors.insert("T1", 23.7);
  sensors.insert("T2", 24.1);
  sensors.insert("P1", 1013.2);
  match sensors.get("T1") {
    Some(value) => println!("T1 = {value}"),
    None => println!("sensor not found"),
  }
  for (id, value) in &sensors {
    println!("{id}: {value}");
} }
```

```
HashMap<String, f64>
           │       └─ value type
           └──────── key type
```

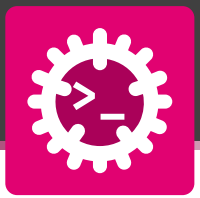


h() hash function

Typical use cases

- Lookup tables
- Configuration values
- Sensor id to value
- User id to user data
- Counting occurrences

> A HashMap is fast, but iteration order is not sorted.



A common real-world pattern - `entry()` is useful when a key may or may not already exist.

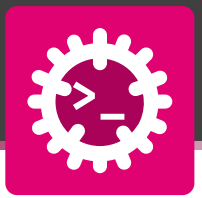
```rust
use std::collections::HashMap;

fn main() {
  let events = [
    "open", "close", "open",
    "error", "open", "error",
  ];
  let mut counts = HashMap::new(); // new empty HashMap
  for event in events {
    // if the key doesn't exist, insert it with value 0,
    // then get back a mutable reference to the value
    let counter = counts.entry(event).or_insert(0);
    // this modifies the value inside the HashMap
    *counter += 1;
  }
  println!("{counts:?}");
}
```

```
{
  "open": 3,
  "close": 1,
  "error": 2
}
```

Meaning

```rust
counts.entry(event).or_insert(0)
```

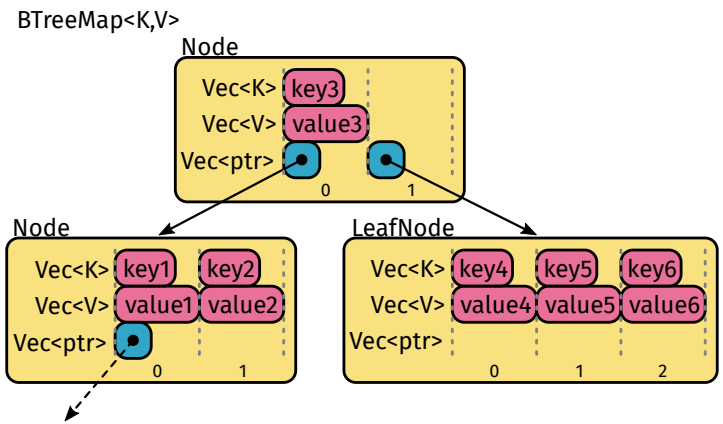
- Get existing counter
- Or insert `0`
- Then increment it via `&mut`



`BTreeMap<K, V>` **Sorted key-value lookup** - stores key-value pairs sorted by key.

```rust
use std::collections::BTreeMap;

fn main() {
  let mut measurements = BTreeMap::new();

  measurements.insert("10:00", 23.7);
  measurements.insert("09:00", 22.9);
  measurements.insert("11:00", 24.1);

  for (time, value) in &measurements {
    println!("{time}: {value}");
  }
}
```

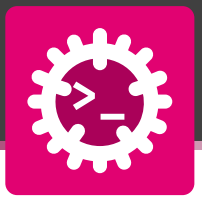
```
09:00: 22.9
10:00: 23.7
11:00: 24.1
```



Use `BTreeMap` when you need

- Sorted keys
- Ordered iteration
- Range queries
- Predictable output order

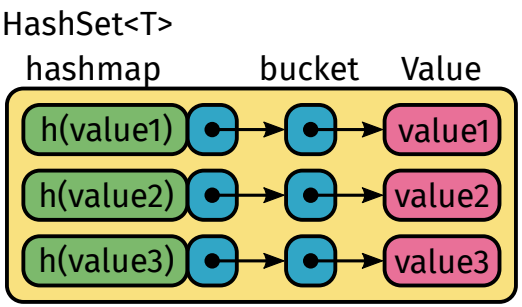
> Use HashMap for fast lookup. Use BTreeMap for ordered data.



Unique values - A HashSet<T> stores each value only once.

```rust
use std::collections::HashSet;

fn main() {
  let mut ids = HashSet::new();

  ids.insert("T1");
  ids.insert("T2");
  ids.insert("T1"); // duplicate

  println!("count = {}", ids.len());

  if ids.contains("T2") {
    println!("T2 exists");
  }
}
```

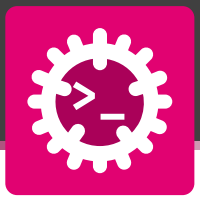
```
count = 2
T2 exists
```



Typical use cases

- Remove duplicates
- Track seen elements
- Membership tests
- Compare groups

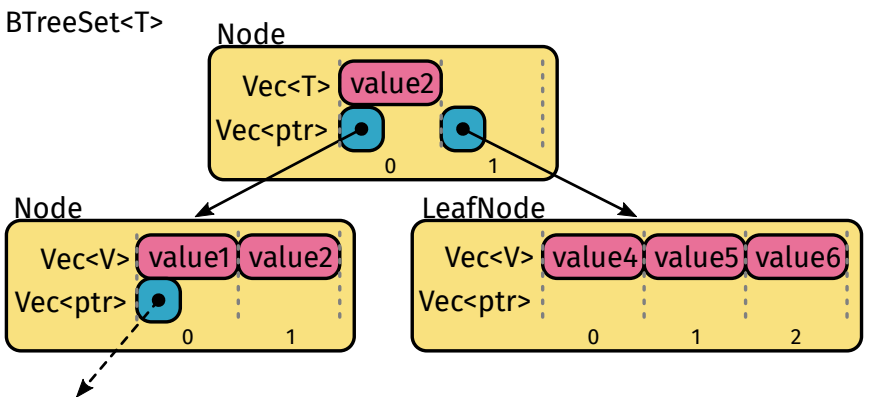
A HashSet<T> is basically a HashMap<T, ()> without values.



Unique and sorted values - A BTreeSet<T> stores unique values in sorted order.

```rust
use std::collections::BTreeSet;

fn main() {
  let mut ids = BTreeSet::new();

  ids.insert("T3");
  ids.insert("T1");
  ids.insert("T2");
  ids.insert("T1");

  for id in &ids {
    println!("{id}");
  }
}
```

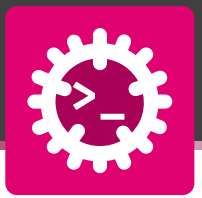
```
T1
T2
T3
```



Use BTreeSet when you need

- Unique values
- Sorted output
- Deterministic iteration
- Range-like operations

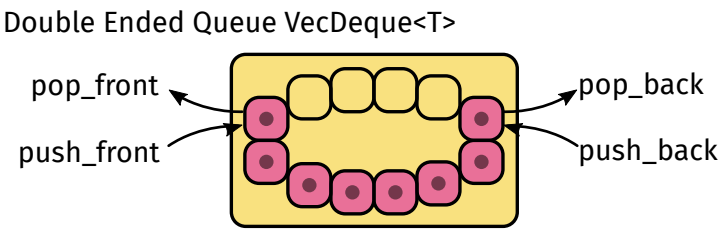
Use HashSet for speed. Use BTreeSet for order.



First in, first out - A queue processes values in the same order as they arrive.

```rust
use std::collections::VecDeque;

fn main() {
  let mut queue = VecDeque::new();

  queue.push_back("read sensor");
  queue.push_back("filter value");
  queue.push_back("send packet");

  while let Some(task) = queue.pop_front() {
    println!("processing: {task}");
  }
}
```

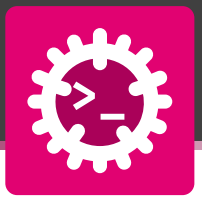
```
processing: read sensor
processing: filter value
processing: send packet
```



Queue operations

`push_front(x)`	add at the front
`push_back(x)`	add at the end
`pop_front()`	remove from the front
`pop_back()`	remove from the back
`front()`	read first item
`back()`	read last item

Which Collection Should I Use?



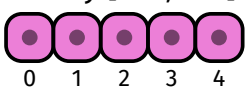
Practical rule of thumb

Need	Use
Fixed number of values	Array `[T; N]`
Growable list	Vector `Vec<T>`
Fast lookup by key	`HashMap<K, V>`
Sorted lookup by key	`BTreeMap<K, V>`
Unique values	`HashSet<T>`
Unique sorted values	`BTreeSet<T>`
FIFO queue	`VecDeque<T>`

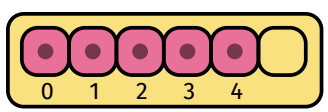
Start with `Vec<T>`. Use maps, sets, and queues when the access pattern requires them.

Sequences

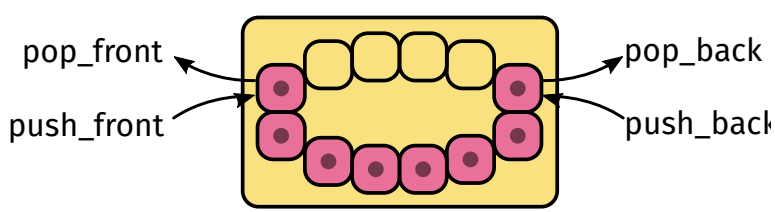
Array [<T>, len]



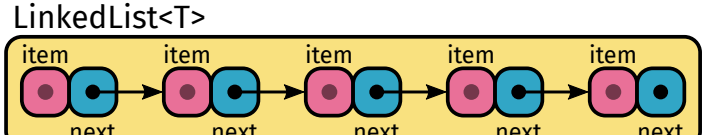
Vector Vec<T>



)ouble Ended Queue VecDeque<T>

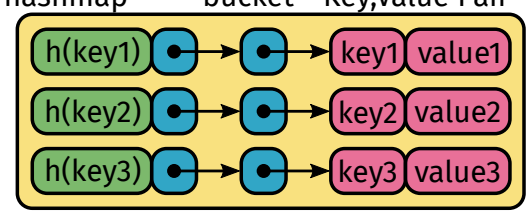


LinkedList<T>



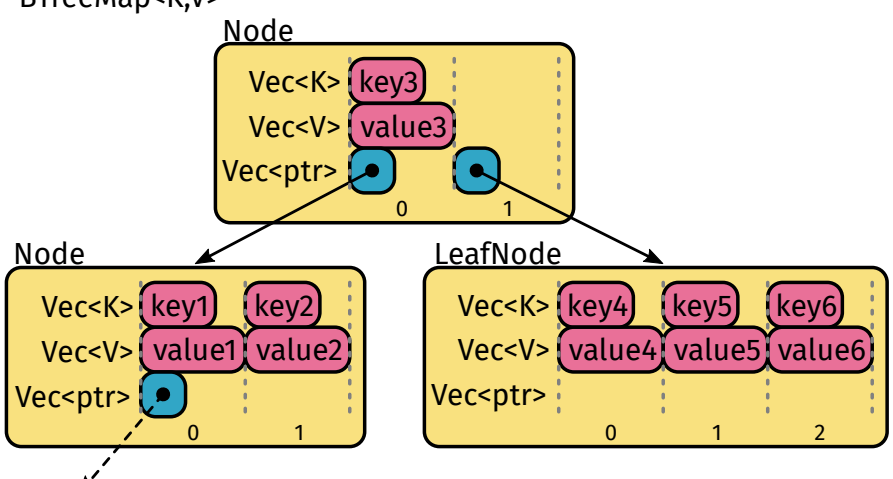
Maps

HashMap<K,V>



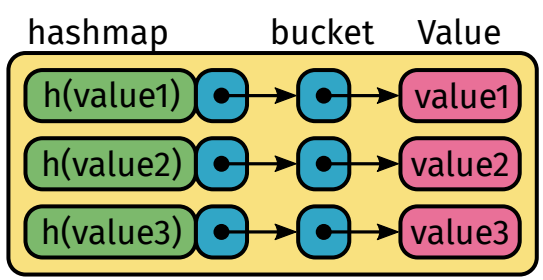
h() hash function

BTreeMap<K,V>

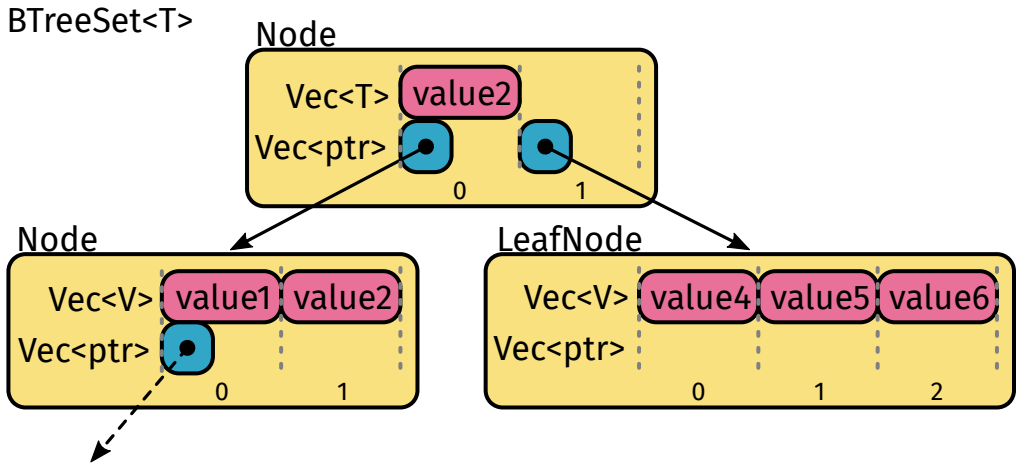


Sets

HashSet<T>

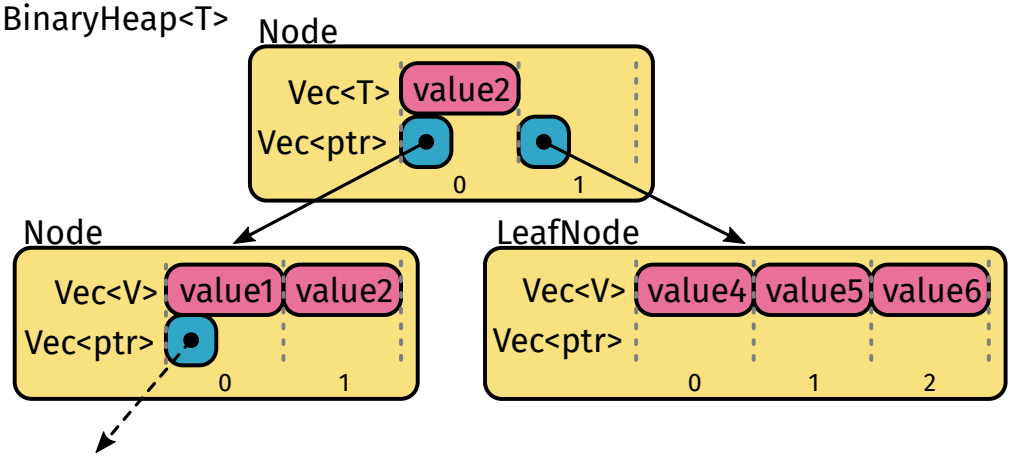


BTreeSet<T>

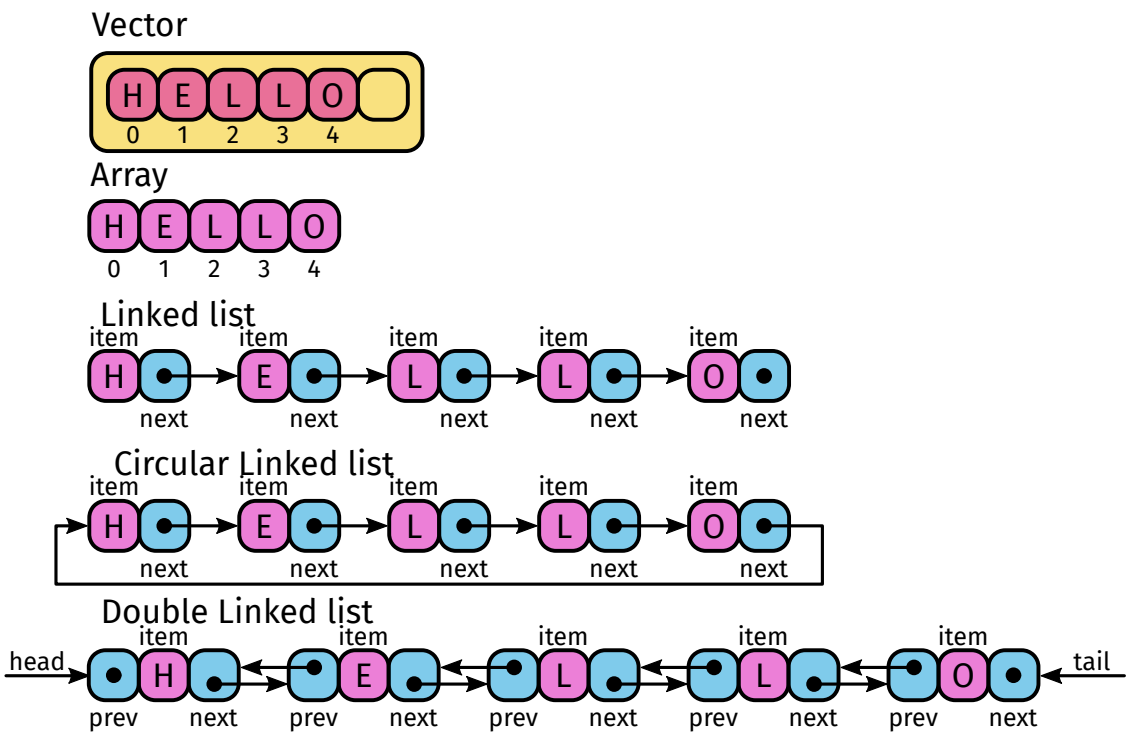


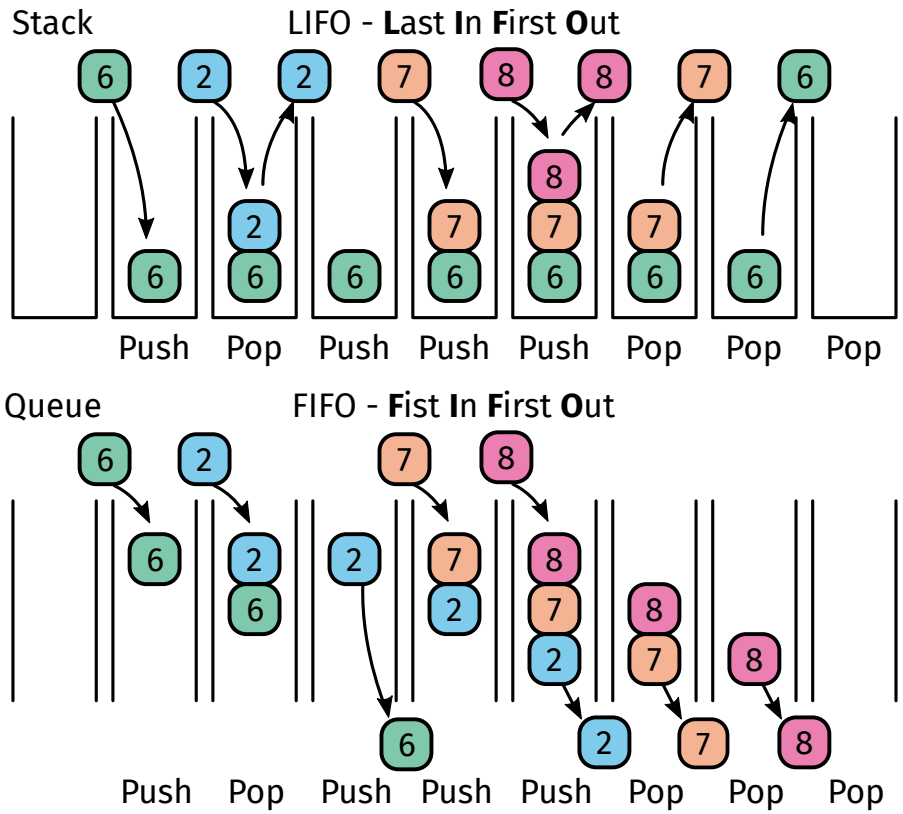
Misc

BinaryHeap<T>



Container Overview





Exercise - Composite Types

Read the program and write the value printed for `a`, `b`, `c` and `d`.

```
use std::collections::HashMap;

fn main() {
  let votes = vec!["red", "blue", "red",
                   "green", "red", "blue"];
  let total = votes.len();

  let mut tally: HashMap<&str, u32> = HashMap::new();
  for v in votes {
    *tally.entry(v).or_insert(0) += 1;
  }

  println!("a {}", total);                  // a
  println!("b {}", tally.len());            // b
  println!("c {}", tally["red"]);           // c
  println!("d {:?}", tally.get("yellow")); // d
}
```

`entry(k).or_insert(0)` returns a `&mut` to the value, inserting `0` first if the key is new.

`tally.get(k)` returns an `Option`, while `tally[k]` panics on a missing key.

QUESTIONS

CAN BE ASKED BY ANYONE ANYTIME

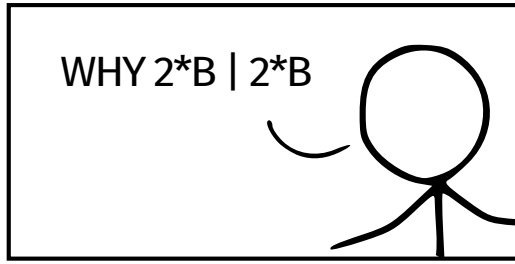
WHY DO WHALES JUMP
WHY ARE THERE MIRRORS ABOVE BEDS
WHY DO I SAY UH
WHY IS SEA SALT BETTER
WHY ARE THERE TREES IN THE MIDDLE OF FIELDS
WHY IS THERE NOT A POKEMON MMO
WHY IS THERE LAUGHING IN TV SHOWS
WHY ARE THERE DOORS ON THE FREEWAY
WHY ARE THERE SO MANY SVCHOST.EXE RUNNING
WHY AREN'T ANY COUNTRIES IN ANTARCTICA
WHY ARE THERE SCARY SOUNDS IN MINECRAFT
WHY IS THERE KICKING IN MY STOMACH
WHY ARE THERE TWO SLASHES AFTER HTTP
WHY ARE THERE CELEBRITIES
WHY DO SNAKES EXIST
WHY DO OYSTERS HAVE PEARLS
WHY ARE DUCKS CALLED DUCKS
WHY DO THEY CALL IT THE CLAP
WHY ARE KYLE AND CARTMAN FRIENDS
WHY IS THERE AN ARROW ON AANG'S HEAD
WHY ARE TEXT MESSAGES BLUE
WHY ARE THERE MUSTACHES ON CLOTHES
WHY WUBA LUBBA DUB DUB MEANING
WHY IS THERE A WHALE AND A POT FALLING
WHY ARE THERE SO MANY BIRDS IN SWISS
WHY IS THERE SO LITTLE RAIN IN WALLIS
WHY IS WALLIS WEATHER FORECAST ALWAYS WRONG

WHY ARE HATS SO EXPENSIVE
WHY IS THERE CAFFEINE IN MY SHAMPOO
WHY HAVE DINOSAURS NO FUR

WHY DO TWINS HAVE DIFFERENT FINGERPRINTS
WHY ARE SWISS AFRAID OF DRAGONS

WHY IS HTTPS CROSSED OUT IN RED
WHY IS THERE A LINE THROUGH HTTPS
WHY IS THERE A RED LINE THROUGH HTTPS ON TWITTE
WHY IS HTTPS IMPORTANT

WHY IS YKK ON ALL ZIPPERS



WHY DO IGUANAS DIE

WHY AREN'T THERE DINOSAUR GHOSTS

WHY AREN'T ECONOMISTS RICH
WHY DO AMERICANS CALL IT SOCCER
WHY ARE MY EARS RINGING
WHY IS 42 THE ANSWER TO EVERYTHING
WHY CAN'T NOBODY ELSE LIFT THORS HAMMER
WHY IS MARVIN ALWAYS SO SAD

WHY IS THERE A SWARM OF ANTS
WHY IS THERE PILEGRIM

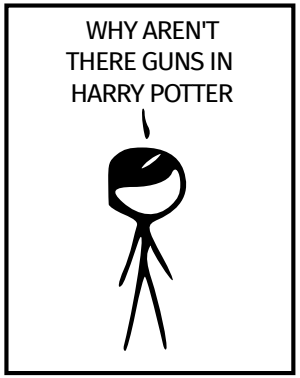
WHY IS THERE LAVA

WHY ARE THERE SO MANY CROWS IN ROCHESTER
WHY IS TO BE OR NOT TO BE FUNNY
WHY DO CHILDREN GET CANCER
WHY IS POSEIDON ANGRY WITH ODYSSEUS
WHY IS THERE ICE IN SPACE

WHY ARE THERE ANTS IN MY LAPTOP

WHY IS EARTH TILTED
WHY IS SPACE BLACK
WHY IS OUTER SPACE SO COLD
WHY ARE THERE PYRAMIDS ON THE MOON
WHY IS NASA SHUTTING DOWN

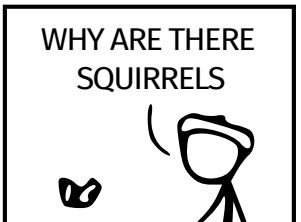
WHY ARE THERE FEMALE M



WHY IS THERE AN OWL IN MY BACKYARD
WHY IS THERE AN OWL OUTSIDE MY WINDOW
WHY IS THERE AN OWL ON THE DOLLAR BILL
WHY DO OWLS ATTACK PEOPLE
WHY ARE FPGA's EVERYWHERE
WHY ARE THERE HELICOPTERS CIRCLING MY HOUSE
WHY ARE THERE GODS
WHY ARE THERE TWO SPOCKS
WHAT IS https://xkcd.com/1256/
WHY DO THEY SAY T-MINUS
WHY ARE OCEANS BECOMMING MORE ACIDIC

WHY ARE THERE MALE AND FEMALE BIKES

WHY ARE THERE BRIDESMAIDS
WHY DO DYING PEOPLE REACH UP
HOW FAST IS LIGHTSPEED
WHY ARE OLD KLINGONS DIFFERENT

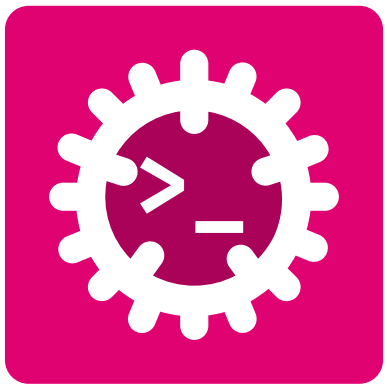


WHY IS THERE HE

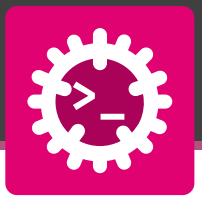
WHY ARE THERE TINY SPIDERS IN MY HOUSE
WHY DO SPIDERS COME INSIDE
WHY ARE THERE HUGE SPIDERS IN MY HOUSE
WHY ARE THERE LOTS OF SPIDERS IN MY HOUSE
WHY ARE THERE SPIDERS IN MY ROOM
WHY ARE THERE SO MANY SPIDERS IN MY ROOM
WHY DO SPYDER BITES ITCH

WHY ARE MY BOOBS ITCHY
WHY ARE CIGARETTES LEGAL
WHY ARE THERE DUCKS IN MY POOL
WHY IS JESUS WHITE
WHY IS THERE LIQUID IN MY EAR
WHY DO Q TIPS FEEL GOOD
WHY DO PEOPLE DIE

WHY IS LIFE



Glossary & Bibliography



Computer Science

***ASCII* – American Standard Code for Information Interchange**: Character encoding standard for representing text. 29

***RGB* – Red Green Blue**: Color model representing colors as combinations of red, green, and blue. 29

Data Structure

***FIFO* – First In First Out**: Queue discipline where the oldest element is processed first. 105, 116

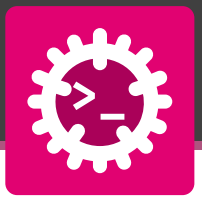
Programming

***heap* – heap**: Memory area used for dynamic memory allocation. 48

***stack* – stack**: Memory area used for static memory allocation and function call management. 13

***USV* – Unicode Scalar Value**: A Unicode code point that is not a surrogate and represents a single character. 46

***UTF* – Unicode Transformation Format**: Encoding standard for representing Unicode characters. 48



Bibliography

[1] S. Klabnik, C. Nichols, and C. Krycho, “The Rust Programming Language.”

[2] J. Blandy, J. Orendorff, and L. F. s Tindall, *Programming Rust: Fast, Safe Systems Development*. O'Reilly Media, Incorporated, 2021.

[3] H. Collard, *Black Hat Rust*, vol. 1. Amazon, 2025.

[4] A. Shvets, “Refactoring Guru.” 2026.

[5] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*. Pearson Education, 2008.

[6] “SVG Repo - Free SVG Vectors and Icons.”